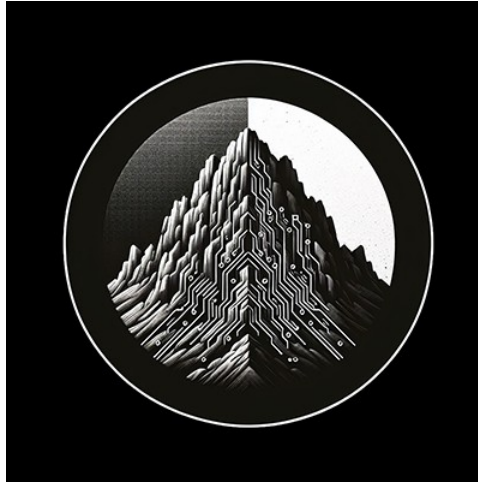




RTL Design Sherpa

RTL Clock Domain Crossing Module Reference — Rev 0.90

July 2026

Table of Contents

1 Clock Domain Crossing (CDC).....	20
1.1 Choosing a technique.....	20
1.1.1 Quick reference.....	21
1.2 Reset Considerations.....	22
1.2.1 Why encoding decides reset behavior.....	22
1.2.2 Async FIFO reset behavior.....	25
1.2.3 Silicon evidence.....	26
1.2.4 Safe usage checklist for 2-phase.....	26
1.2.5 Recovering an existing design.....	26
1.3 cdc_synchronizer.....	27
1.4 cdc_open_loop.....	28
1.5 cdc_2_phase_handshake.....	29
1.5.1 Shared handshake interface.....	30
1.5.2 Theory of operation.....	30
1.5.3 Swapping between variants.....	31
1.6 cdc_4_phase_handshake.....	31
1.6.1 Theory of operation.....	31
1.7 Async FIFO Pointers (Gray and Johnson).....	32
1.7.1 Gray vs Johnson encoding.....	32
1.7.2 Sizing the depth.....	34
1.7.3 Walkthrough: is an even-depth FIFO worth it at 512 bits?.....	35
1.8 SDC constraints.....	39

1.9 Common mistakes.....	40
1.10 Verification status.....	41
1.11 Module reference.....	41
1.12 Navigation.....	42
2 Glitch-Free N-DFF Synchronizer (glitch_free_n_dff_arn.sv).....	42
2.1 Purpose.....	42
2.2 Ports.....	43
2.2.1 Input Ports.....	43
2.2.2 Output Ports.....	43
2.2.3 Parameters.....	43
2.3 Clock Domain Crossing Theory.....	43
2.3.1 Metastability Problem.....	43
2.3.2 Metastability Effects.....	43
2.4 Multi-Stage Synchronizer Solution.....	44
2.4.1 Stage-by-Stage Analysis.....	44
2.5 Implementation Details.....	44
2.5.1 Packed Array Structure.....	44
2.5.2 Shift Register Behavior.....	44
2.5.3 Reset Behavior.....	45
2.6 MTBF (Mean Time Between Failures) Analysis.....	45
2.6.1 MTBF Calculation.....	45
2.6.2 Stage Count Impact.....	45
2.6.3 Practical Guidelines.....	46
2.7 Latency vs. Reliability Trade-off.....	46

2.7.1 Synchronizer Latency.....	46
2.7.2 Performance Impact.....	46
2.8 Usage Guidelines.....	46
2.8.1 Appropriate Use Cases.....	46
2.8.2 Inappropriate Use Cases.....	46
2.8.3 Multi-Bit Considerations.....	47
2.9 Advanced Applications.....	47
2.9.1 Reset Synchronizer Pattern.....	47
2.9.2 Enable Signal Synchronization.....	47
2.9.3 Status Flag Collection.....	47
2.10 Synthesis Considerations.....	48
2.10.1 FPGA Implementation.....	48
2.10.2 Timing Constraints.....	48
2.10.3 Optimization Prevention.....	48
2.11 Verification Strategies.....	48
2.11.1 Functional Verification.....	48
2.11.2 Metastability Testing.....	48
2.11.3 Formal Verification.....	48
2.12 Debug Features.....	49
2.12.1 Waveform Analysis.....	49
2.12.2 Simulation Visibility.....	49
2.13 Related Modules.....	49
2.14 Navigation.....	49
3 Binary to Gray Code Converter.....	49

3.1 Overview.....	49
3.2 Module Declaration.....	50
3.3 Parameters.....	50
3.3.1 WIDTH.....	50
3.4 Ports.....	50
3.4.1 Inputs.....	50
3.4.2 Outputs.....	50
3.5 Gray Code Theory.....	50
3.5.1 Mathematical Definition.....	50
3.5.2 Why Gray Code?.....	51
3.5.3 Gray Code Sequence Examples.....	51
3.6 Implementation Analysis.....	52
3.6.1 Core Logic.....	52
3.6.2 Logic Structure.....	52
3.6.3 Gate-Level Implementation.....	52
3.6.4 Timing Characteristics.....	53
3.7 Design Examples and Applications.....	53
3.7.1 1. Asynchronous FIFO Pointer.....	53
3.7.2 2. Clock Domain Crossing Counter.....	54
3.7.3 3. Rotary Encoder Interface.....	55
3.7.4 4. Address Scrambling for Memory Testing.....	56
3.7.5 5. Multi-Bit Synchronizer with Gray Code.....	57
3.8 Companion Gray-to-Binary Converter.....	58
3.8.1 Implementation.....	58

3.8.2 Gray-to-Binary Truth Table (4-bit).....	59
3.9 Advanced Implementations.....	59
3.9.1 1. Parameterized Converter with Validation.....	59
3.9.2 2. Pipelined Converter for High Speed.....	60
3.9.3 3. Bidirectional Converter.....	61
3.10 Verification and Testing.....	62
3.10.1 Comprehensive Test Bench.....	62
3.10.2 Property-Based Verification.....	63
3.10.3 Coverage Model.....	65
3.11 Synthesis Optimization.....	65
3.11.1 Resource Utilization.....	65
3.11.2 Timing Optimization.....	65
3.11.3 Power Optimization.....	66
3.12 Common Applications Summary.....	67
3.12.1 1. Asynchronous FIFOs: Prevent metastability in pointer comparisons	67
3.12.2 2. Clock Domain Crossing: Safe multi-bit signal transfer.....	67
3.12.3 3. Position Encoders: Mechanical/optical encoder interfaces.....	67
3.12.4 4. Memory Address Generation: Reduce EMI and power spikes.....	67
3.12.5 5. Test Pattern Generation: Controlled single-bit transitions.....	67
3.12.6 6. Error Detection: Single-bit error detection schemes.....	67
3.12.7 7. ADC/DAC Interfaces: Reduce glitches in conversion systems.....	67
3.13 Design Guidelines.....	67

3.13.1 1. Always Use for Async Boundaries: Gray code is essential for safe asynchronous transfers.....	67
3.13.2 2. Consider Timing: Purely combinational - no setup/hold concerns...	67
3.13.3 3. Verify Single-Bit Property: Always verify adjacent values differ by one bit.....	67
3.13.4 4. Plan for Back-Conversion: Usually need Gray-to-Binary converter too.....	67
3.13.5 5. Check Width Requirements: Ensure sufficient bits for application range.....	67
3.14 Navigation.....	68
4 Gray-to-Binary Converter (gray2bin.sv).....	68
4.1 Purpose.....	68
4.2 Ports.....	68
4.2.1 Input Ports.....	68
4.2.2 Output Ports.....	68
4.2.3 Parameters.....	68
4.3 Gray Code Theory.....	68
4.3.1 What is Gray Code?.....	68
4.3.2 Why Gray Codes Matter for CDC.....	69
4.4 Conversion Algorithm.....	69
4.4.1 Mathematical Foundation.....	69
4.4.2 Implementation.....	69
4.4.3 Bit-by-Bit Breakdown.....	69
4.5 Functional Examples.....	69
4.5.1 4-Bit Conversion Table.....	69

4.5.2 Step-by-Step Example (Gray 0110 → Binary).....	70
4.6 Implementation Analysis.....	70
4.6.1 XOR Reduction Technique.....	70
4.6.2 Generate Loop Benefits.....	71
4.7 Use Cases in Digital Design.....	71
4.7.1 Asynchronous FIFO Pointers.....	71
4.7.2 Clock Domain Crossing Counters.....	71
4.7.3 Position Encoding.....	71
4.8 Timing Characteristics.....	71
4.8.1 Propagation Delay.....	71
4.8.2 Delay Analysis by Width.....	72
4.8.3 Synthesis Optimization.....	72
4.9 Design Considerations.....	72
4.9.1 Width Scaling.....	72
4.9.2 Input Validation.....	72
4.9.3 Pipeline Considerations.....	72
4.10 Verification Strategies.....	73
4.10.1 Exhaustive Testing.....	73
4.10.2 Property-Based Verification.....	73
4.10.3 Random Testing.....	73
4.11 Common Mistakes and Pitfalls.....	73
4.11.1 Bit Order Confusion.....	73
4.11.2 Width Mismatches.....	73
4.11.3 Timing Assumptions.....	74

4.12 Related Modules.....	74
4.13 Navigation.....	74
5 Johnson-to-Binary Converter (johnson2bin.sv).....	74
5.1 Purpose.....	74
5.2 Ports.....	75
5.2.1 Input Ports.....	75
5.2.2 Output Ports.....	75
5.2.3 Parameters.....	75
5.3 Johnson Counter Sequence Review.....	75
5.3.1 Johnson Counter Characteristics.....	75
5.3.2 Key Properties.....	75
5.4 Conversion Algorithm.....	76
5.4.1 Strategy Overview.....	76
5.4.2 Position Detection Module.....	76
5.5 Detailed Conversion Examples.....	76
5.5.1 First Half Conversion (MSB = 0).....	76
5.5.2 Second Half Conversion (MSB = 1).....	76
5.5.3 Special Cases.....	76
5.6 Implementation Deep Dive.....	77
5.6.1 Three-Part Binary Construction.....	77
5.6.2 Width Calculations.....	77
5.6.3 Why This Algorithm Works.....	77
5.7 Use in Asynchronous FIFO.....	77
5.7.1 Context in FIFO Operation.....	77

5.7.2 Why Clock is Required.....	78
5.8 Comparison with Standard Gray2Bin.....	78
5.9 Performance Characteristics.....	78
5.9.1 Timing Analysis.....	78
5.9.2 Resource Utilization.....	78
5.10 Design Considerations.....	79
5.10.1 When to Use Johnson vs. Gray.....	79
5.10.2 Resource Planning.....	79
5.10.3 Verification Challenges.....	79
5.11 Error Conditions and Debug.....	79
5.11.1 Invalid Johnson Sequences.....	79
5.11.2 Debug Visibility.....	79
5.11.3 Common Issues.....	80
5.12 Related Modules.....	80
5.13 Advanced Topics.....	80
5.13.1 Hierarchical Johnson Counters.....	80
5.13.2 Alternative Position Detection.....	80
5.14 Navigation.....	80
6 Binary-Gray Counter Module.....	80
6.1 Overview.....	80
6.2 Module Declaration.....	81
6.3 Parameters.....	81
6.3.1 WIDTH.....	81
6.4 Ports.....	81

6.4.1 Inputs.....	81
6.4.2 Outputs.....	81
6.5 Architecture Details.....	82
6.5.1 Internal Logic.....	82
6.5.2 Gray Code Conversion.....	82
6.6 Gray Code Theory.....	82
6.6.1 Why Gray Code?.....	82
6.6.2 Gray Code Sequence Example (4-bit).....	82
6.7 Implementation Analysis.....	83
6.7.1 Combinational Logic.....	83
6.7.2 Sequential Logic.....	83
6.8 Timing Characteristics.....	83
6.8.1 Critical Paths.....	83
6.8.2 Propagation Delays.....	84
6.9 Application: Asynchronous FIFO.....	84
6.9.1 FIFO Pointer Implementation.....	84
6.9.2 Cross-Domain Synchronization.....	84
6.9.3 FIFO Status Generation.....	85
6.10 Advanced Features.....	85
6.10.1 Almost Full/Empty Flags.....	85
6.10.2 Gray-to-Binary Conversion Function.....	85
6.11 Design Examples.....	86
6.11.1 1. 8-bit Asynchronous FIFO Pointer.....	86
6.11.2 2. Clock Domain Crossing Counter.....	86

6.12 Verification Strategy.....	86
6.12.1 Test Scenarios.....	86
6.12.2 Coverage Points.....	87
6.12.3 Assertions.....	87
6.13 Synthesis Considerations.....	88
6.13.1 Resource Utilization.....	88
6.13.2 Optimization Techniques.....	88
6.13.3 Tool-Specific Attributes.....	88
6.14 Performance Characteristics.....	88
6.14.1 Maximum Frequency.....	88
6.14.2 Power Consumption.....	88
6.15 Common Applications.....	88
6.16 Troubleshooting Guide.....	89
6.16.1 Common Issues.....	89
6.16.2 Debug Techniques.....	89
6.17 WaveDrom Visualization.....	89
6.17.1 Scenario 1: Binary vs Gray Code Comparison.....	89
6.17.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE.....	89
6.17.3 Scenario 3: Lookahead Signal (counter_bin_next).....	90
6.17.4 Scenario 4: Enable and Reset Control.....	90
6.18 Test and Verification.....	91
6.19 Navigation.....	91
7 Clock Pulse Generator - Comprehensive Documentation.....	92
7.1 Overview.....	92

7.2 Module Declaration.....	92
7.3 Parameters.....	92
7.3.1 WIDTH.....	92
7.4 Ports.....	92
7.4.1 Inputs.....	92
7.4.2 Outputs.....	93
7.5 Architecture and Implementation.....	93
7.5.1 Internal Counter.....	93
7.5.2 Core Logic.....	93
7.5.3 Operation Principles.....	93
7.5.4 Timing Characteristics.....	93
7.6 Timing Diagrams.....	94
7.6.1 Basic Operation (WIDTH=4).....	94
7.6.2 Reset Behavior.....	94
7.6.3 Different WIDTH Values.....	94
7.7 Design Examples and Applications.....	94
7.7.1 1. Heartbeat and Status Indicators.....	94
7.7.2 2. Sampling and Data Acquisition.....	95
7.7.3 3. Communication Protocol Timing.....	97
7.7.4 4. Memory Refresh Controller.....	98
7.7.5 5. Multi-Rate Pulse Generation.....	99
7.7.6 6. Test Pattern Generation.....	100
7.8 Advanced Features and Variations.....	102
7.8.1 1. Enable-Controlled Pulse Generator.....	102

7.8.2 2. Variable Width Pulse Generator.....	103
7.8.3 3. Multi-Phase Pulse Generator.....	104
7.8.4 4. Burst Pulse Generator.....	105
7.9 Verification and Testing.....	107
7.9.1 Comprehensive Test Bench.....	107
7.9.2 Coverage Model.....	109
7.9.3 Formal Properties.....	110
7.10 Synthesis Considerations.....	111
7.10.1 Resource Utilization.....	111
7.10.2 Timing Optimization.....	111
7.11 Common Applications Summary.....	112
7.12 Design Guidelines.....	112
7.13 Navigation.....	112
8 APB Slave CDC.....	113
8.1 Overview.....	113
8.1.1 Key Features.....	113
8.2 Module Interface.....	113
8.3 Parameters.....	114
8.4 Clock Domains.....	115
8.4.1 APB Domain (pclk).....	115
8.4.2 AXI Domain (aclk).....	115
8.5 CDC Implementation.....	115
8.5.1 Structure.....	115
8.5.2 Maximum Clock Ratio.....	116

8.5.3 Reset Behavior.....	116
8.5.4 Why Not the Previous 2-Phase Handshake.....	116
8.5.5 Timing Constraints.....	117
8.6 Waveforms.....	117
8.6.1 Scenario 1: Write Transaction with CDC.....	117
8.6.2 Scenario 2: Read Transaction with CDC.....	117
8.6.3 Scenario 3: Back-to-Back Writes with CDC.....	118
8.6.4 Scenario 4: Back-to-Back Reads with CDC.....	118
8.6.5 Scenario 5: Write-to-Read Transition with CDC.....	119
8.6.6 Scenario 6: Read-to-Write Transition with CDC.....	119
8.6.7 Scenario 7: Error Response with CDC.....	120
8.7 Usage Example.....	120
8.8 References.....	121
8.9 Navigation.....	122
9 APB Slave with CDC (Clock-Gated).....	122
9.1 Quick Reference.....	122
9.2 Summary.....	122
9.3 Additional Parameters.....	123
9.4 Additional Ports.....	123
9.5 Quick Usage.....	124
9.6 Documentation.....	124
9.7 Navigation.....	125
10 APB5 Slave (Clock Domain Crossing).....	125
10.1 Overview.....	125

10.1.1 Key Features.....	125
10.2 Parameters.....	126
10.3 Ports.....	127
10.3.1 Clock and Reset.....	127
10.3.2 APB5 Slave Interface.....	127
10.3.3 Backend Interface.....	127
10.4 Clock Domain Crossing.....	128
10.4.1 Timing Considerations.....	128
10.5 Usage Example.....	128
10.6 Design Notes.....	129
10.6.1 CDC Mechanism.....	129
10.6.2 FIFO Depth Sizing.....	130
10.6.3 Reset Synchronization.....	130
10.6.4 Latency.....	130
10.7 Related Documentation.....	130
10.8 Navigation.....	131
11 APB5 Slave (CDC + Clock-Gated).....	131
11.1 Overview.....	131
11.1.1 Key Features.....	131
11.2 Parameters.....	132
11.3 Ports.....	133
11.3.1 Clock and Reset.....	133
11.3.2 Clock Gating Configuration.....	133
11.3.3 APB5 Slave Interface.....	133

11.3.4 Backend Interface.....	134
11.3.5 Status Outputs.....	134
11.4 Clock Gating and CDC Interaction.....	135
11.4.1 Timing Considerations.....	135
11.5 Usage Example.....	136
11.6 Design Notes.....	137
11.6.1 Power and Latency Trade-offs.....	137
11.6.2 Reset Considerations.....	138
11.7 Related Documentation.....	139
11.8 Navigation.....	139

List of Tables

Table 1: CDC building blocks at a glance.....	20
Table 2: CDC technique quick reference by scenario.....	21
Table 3: How pointer encoding determines reset behavior.....	22
Table 4: 2-phase handshake reset domains per flop.....	22
Table 5: Options for recovering a design using 2-phase across independent resets	26
Table 6: cdc_synchronizer parameters.....	27
Table 7: cdc_open_loop parameters.....	28
Table 8: cdc_open_loop ports.....	29
Table 9: cdc_2_phase_handshake parameters.....	29
Table 10: Shared handshake interface ports.....	30
Table 11: Consequences of swapping handshake variants.....	31
Table 12: cdc_4_phase_handshake parameters.....	31
Table 13: Async FIFO pointer encodings.....	32
Table 14: 3-bit Gray vs the equivalent 4-bit Johnson counter, full cycle. “flips bit#” is a bit POSITION; “Ham. dist” is the number of bits that changed, which is 1 everywhere including the wrap.....	33
Table 15: Pointer width by encoding – Gray is logarithmic, Johnson is linear.....	34
Table 16: Worked depth-sizing example.....	35
Table 17: Nominal storage overhead of Gray depth rounding by data width.....	35
Table 18: 512-bit walkthrough: depth after applying the encoding constraint.....	36
Table 19: 512-bit walkthrough: nominal storage delta.....	36
Table 20: 512-bit walkthrough: pointer flop cost.....	36
Table 21: 512-bit walkthrough: whether the saving is real, by memory style.....	37
Table 22: Depth-36 case: storage delta when Gray must round 36 up to 64.....	38
Table 23: Depth-36 case: pointer flop cost.....	38
Table 24: Depth-36 case: ASIC memory styles, all of which accept an even depth.....	39
Table 25: CDC module reference.....	41

List of Waveforms

Waveform 1.1: 2-Phase Normal Transfer.....	23
Waveform 1.2: 2-Phase Hazard - Destination Reset Alone.....	23
Waveform 1.3: 4-Phase Normal Transfer.....	24
Waveform 1.4: 4-Phase One-Sided Reset Is Safe.....	25

1 Clock Domain Crossing (CDC)

RTL: rtl/amba/cdc/ **Filelists:** rtl/amba/filelists/cdc_*.f **Status:** Production Ready **for common-clock, common-reset verification.** Reset behavior across independent domains is argued from the RTL and, for the 2-phase hazard, confirmed on silicon – but it is **not** covered by the formal proofs, which drive both domains from one clock and one reset. See [Verification status](#) before relying on that badge.

This is the single reference for clock domain crossing in this repository. It covers every CDC building block, how to choose between them, and the reset behavior that distinguishes them – which is the property most likely to bite you and the least likely to show up in simulation.

CDC building blocks at a glance

Module	Category	One-line
cdc_synchronizer	Multi-flop sync	N-stage synchronizer for quasi-static signals
cdc_open_loop	Open loop	Source holds data + valid, no acknowledge
cdc_2_phase_handshake	Closed loop	Toggle (NRZ) valid/ready handshake, faster
cdc_4_phase_handshake	Closed loop	Level (RZ) valid/ready handshake, classic
fifo_async / gaxi_fifo_async	Pointer sync	Streaming data, buffered

1.1 Choosing a technique

Need to cross clock domains with multi-bit data?

```
|
+-- Is data streaming / continuous?
|   |
|   +-- YES --> Async FIFO (fifo_async or gaxi_fifo_async)
|               Sustained 1-per-clock throughput, buffered
|   |
|   +-- NO, occasional transfers -->
|       |
|       +-- Does the source need backpressure (flow control)?
|           |
|           +-- YES --> Closed-loop handshake
|                           (cdc_2_phase_handshake or
cdc_4_phase_handshake)
```

```

|
|      |-- NO, source can guarantee hold time -->
|      |
|      |-- Open-loop (cdc_open_loop)
|          Simplest, lowest area, no ack needed
|
Single-bit signal?
|
+-- Level / quasi-static --> cdc_synchronizer or glitch_free_n_dff_arn
+-- Single-cycle pulse    --> sync_pulse
+-- Reset signal          --> reset_sync

```

Then check the **reset rule** before committing to a handshake. It eliminates one of the two handshake variants outright in a common situation, and it is not visible from the port list.

1.1.1 Quick reference

CDC technique quick reference by scenario

Scenario	Module	Why
Config register update (CPU -> peripheral)	cdc_open_loop	Infrequent, no backpressure needed
Interrupt signal (1 bit, pulse)	sync_pulse	Single-cycle event
Status register read (slow -> fast)	cdc_synchronizer	Quasi-static level
APB slave in different clock domain	apb_slave_cdc	Full protocol CDC
Monitor packet crossing	cdc_4_phase_handshake	Occasional, needs flow control
DMA data stream	fifo_async	High throughput, continuous
AXI channel crossing	gaxi_skid_buffer_async	Pipelined, GAXI protocol
Reset distribution	reset_sync	Async assert, sync deassert

1.2 Reset Considerations

DESIGN RULE: If the two clock domains can be reset **independently**, do not use `cdc_2_phase_handshake`. Use `cdc_4_phase_handshake` or an async FIFO. A toggle handshake stores transfer state as **parity**, and parity cannot survive a one-sided reset.

“Independently” includes cases that are easy to miss: a soft reset that clears a datapath but not a register block, a per-block reset, separate power domains, or a reset synchronizer whose deassertion is gated differently on each side.

1.2.1 Why encoding decides reset behavior

How pointer encoding determines reset behavior

	2-phase	4-phase	Async FIFO
Transfer state	Parity (toggle) – relative	Level – absolute	Pointer position – absolute
Idle has a value?	No	Yes (req=0)	Yes (pointers equal)
One-sided reset from idle	Fabricates a transfer	Nothing happens	Reads empty
One-sided reset mid-transfer	Duplicate or lost transfer	Duplicate transfer	Entry may be re-read or dropped

The 2-phase protocol signals “a transfer happened” with a *transition*. The receiver has no absolute reference – it only knows whether the synchronized toggle differs from its previous sample:

```
assign w_req_event = w_req_sync ^ r_req_sync_d;    //  
cdc_2_phase_handshake.sv:182
```

That XOR is the entire protocol state. Each side resets its own flops:

2-phase handshake reset domains per flop

Flop	Domain	Reset by	After reset
r_req_tog	source	rst_src_n	1'b0 (:190)
r_ack_syn c, r_ack_syn c_d	source	rst_src_n	1'b0 (:173-174)
r_ack_tog	destination	rst_dst_n	1'b0
r_req_syn	destination	rst_dst_n	1'b0 (:250-251)

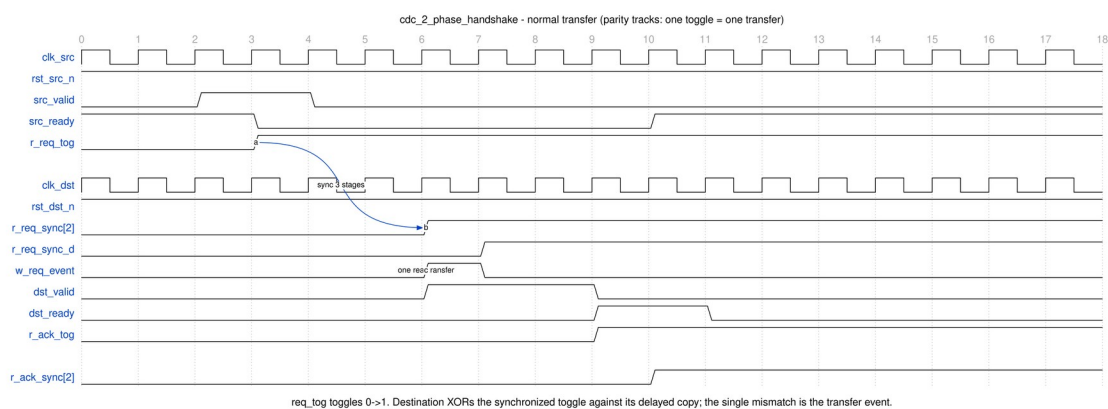
Flop	Domain	Reset by	After reset
------	--------	----------	-------------

c,
r_req_syn
c_d

Reset **both** domains together and everything lands at 0 – parity agrees, no event. Reset **one** and the ends land at opposite parity, and the XOR reports an edge no source transfer produced.

Because parity is relative, there is no toggle value meaning “idle”. Whatever r_req_tog happens to hold, a freshly cleared destination disagrees with it half the time. In 4-phase, req = 0 is idle absolutely: a reset destination observing req = 0 correctly concludes nothing is pending.

1.2.1.1 Waveform 1.1: 2-Phase Normal Transfer

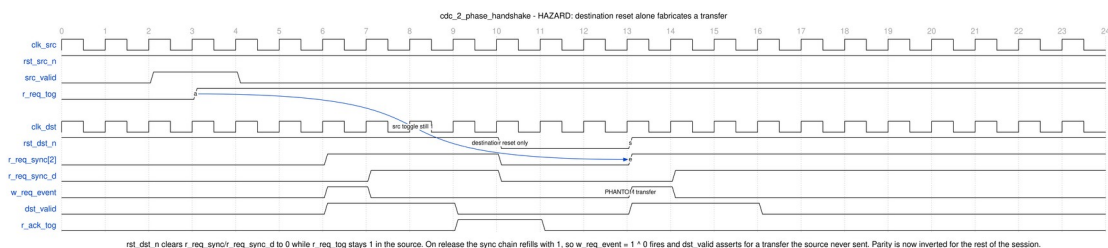


2-phase normal transfer

Source: [cdc2_normal_transfer.json](#)

One src_valid toggles r_req_tog 0 -> 1. Three synchronizer stages later the destination sees the new value, w_req_event pulses for one cycle, dst_valid asserts. Parity now agrees at 1 on both ends.

1.2.1.2 Waveform 1.2: 2-Phase Hazard - Destination Reset Alone



2-phase asymmetric reset hazard

Source: [cdc2_asymmetric_reset_hazard.json](#)

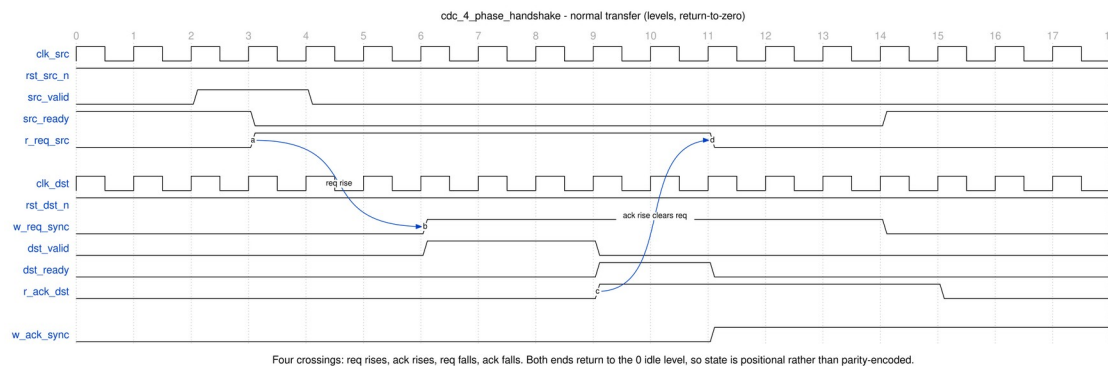
`rst_dst_n` asserts while `rst_src_n` stays high:

1. The destination clears `r_req_sync` and `r_req_sync_d` to 0.
2. The source is untouched – `r_req_tog` is still 1.
3. On release the sync chain refills with 1 while `r_req_sync_d` is the freshly cleared 0.
4. $w_req_event = 1 \wedge 0 = 1$ – a **phantom transfer**. `dst_valid` asserts with stale `r_src_data_hold`, for a transfer the source never sent.

Resetting the source alone behaves the same way. The source FSM is also reset to `S_IDLE` from `S_WAIT_ACK` (: 188 - 190), abandoning an in-flight transfer, and a cleared `r_ack_tog` can present a spurious ack to a source still waiting – so a one-sided reset can **lose** a transfer as well as fabricate one.

Scope of the damage. The handshake itself re-synchronizes: once `r_req_sync_d` catches up to `r_req_tog`, parity agrees again. What is permanent is the **extra item already delivered downstream**. A consumer that samples the latest value self-heals on the next real transfer; a consumer that counts transfers, pushes into a FIFO, or pairs responses to requests positionally is corrupted for the rest of the session.

1.2.1.3 Waveform 1.3: 4-Phase Normal Transfer

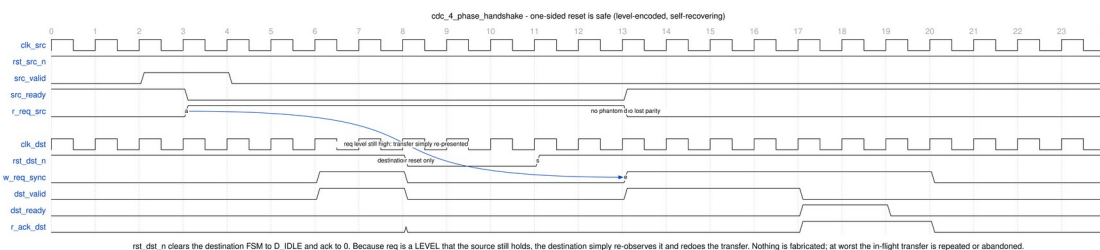


4-phase normal transfer

Source: [cdc4_normal_transfer.json](#)

Four crossings: `r_req_src` rises, `r_ack_dst` rises, `r_req_src` falls, `r_ack_dst` falls. Both ends finish at the 0 idle level.

1.2.1.4 Waveform 1.4: 4-Phase One-Sided Reset Is Safe



4-phase one-sided reset is safe

Source: [cdc4_asymmetric_reset_safe.json](#)

`rst_dst_n` asserts mid-transfer while the source runs:

1. The destination FSM returns to D_IDLE and `r_ack_dst` clears to 0.
2. The source is unaffected, still holding `r_req_src = 1` with data stable.
3. On release the destination re-observes `w_req_sync = 1`, treats it as a fresh request, and redoes the transfer.
4. The source sees the ack and completes normally.

Worst case is one transfer **repeated or dropped**, and only for a transfer genuinely in flight. If the link is idle when the reset lands, the destination sees `req = 0`, reads it as “nothing pending”, and does nothing.

Caveat – duplicate delivery. Step 3 re-delivers a transfer the destination may already have accepted. If the consumer is not idempotent (a FIFO push, a counter increment, a command queue), hold it in reset alongside the destination FSM or drain it afterwards. Level encoding guarantees you never get a transfer the source did not send; it does not guarantee exactly-once delivery.

1.2.2 Async FIFO reset behavior

Gray/Johnson pointers are absolute positions, and each domain resets its own pointer **and its crossed copy of the remote pointer** from the local reset:

```
rd_ptr_gray_cross_inst / rd_ptr_gray2bin_inst -> .rst_n
(axi_wr_aresetn) // local
wr_ptr_gray_cross_inst / wr_ptr_gray2bin_inst -> .rst_n
(axi_rd_aresetn) // local
```

So a one-sided reset leaves that side self-consistent – both pointers zero, reads empty – rather than desynchronized. There is no parity state to invert, so nothing can be fabricated. This is a deliberate design property and the reason an async FIFO is the safe default when reset domains are not shared.

This covers reset ASSERTION, not reset RELEASE. Each domain's reset must still be released synchronously *within that domain* – which is what `reset_sync` (async assert, sync deassert) is for. A raw asynchronous release can violate recovery/removal timing on the pointer flops, and if the two domains release at sufficiently different times the FIFO can begin operating while one side's synchronized copy of the remote pointer is still catching up. That window is benign for an idle FIFO (it reads empty) but not if traffic starts immediately. Hold traffic off until both domains are out of reset, or gate the producer on a released-and-settled indication.

1.2.3 Silicon evidence

This is not theoretical. On the Nexys A7 DDR2 characterization board, an APB CDC using the 2-phase handshake had its core-side reset pulsed by `CTRL.soft_reset` while the APB side stayed up. One phantom transfer entered a slave FSM that pairs responses to requests positionally (`apb_slave.sv` returns whatever response sits at the head of its skid buffer), and **every register read from then on returned the previous register's value** – lagged by ~3 transactions, permanently, until reprogramming. Writes were unaffected, so it presented as a readback bug rather than a CDC bug.

Replacing that handshake with `gaxi_fifo_async` fixed it: reads became stable across `soft_reset`, verified on silicon.

1.2.4 Safe usage checklist for 2-phase

This is a hard stop, not a scorecard. Every condition must hold. Failing any one of them disqualifies the module – there is no partial credit and no mitigation short of changing the crossing or the reset topology.

Use `cdc_2_phase_handshake` only when **all** hold:

- ☐ `rst_src_n` and `rst_dst_n` assert and release together, **or** both derive from one reset no subsystem can pulse independently
- ☐ No soft reset, per-block reset, or power-domain reset touches one side alone
- ☐ The consumer correlates responses to requests by tag, not by position

If any box is unchecked, choose a level-encoded or pointer-encoded crossing.

1.2.5 Recovering an existing design

Options for recovering a design using 2-phase across independent resets

Option	Effort	Notes
Move both domains onto a common reset	Low	Best when the split was accidental
Swap to <code>cdc_4_phase_handshake</code>	Low	Pin-compatible; level encoding self-recovers

Option	Effort	Notes
Swap to gaxi_fifo_async	Medium	Absolute pointers, local-reset robustness, adds buffering

1.3 cdc_synchronizer

Multi-flop synchronizer for **quasi-static** signals. Not a handshake – there is no flow control and no guarantee the destination sees every value.

```

module cdc_synchronizer #(
    parameter int WIDTH      = 1,    // Bus width to synchronize
    parameter int FLOP_COUNT = 3    // Number of synchronizer stages
(2-5)
) (
    input   logic          clk,        // Destination clock
    input   logic          rst_n,      // Asynchronous active-low
reset
    input   logic [WIDTH-1:0] async_in, // Asynchronous input from
source domain
    output  logic [WIDTH-1:0] sync_out // Synchronized output in
destination domain
);

```

cdc_synchronizer parameters

Parameter	Default	Range	Description
WIDTH	1	1+	Number of bits to synchronize
FLOP_COUNT	3	2-5	Synchronizer chain depth

Usage rules

- Input must be **quasi-static**: it changes rarely enough, relative to the destination clock, that missing a transition is acceptable for the application. This is a statistical property, not a hard timing check the synchronizer enforces – it will sample whatever is present. As a sizing rule of thumb, allow at least FLOP_COUNT + 1 destination clocks between transitions; change faster than that and the destination can miss transitions entirely or sample mid-flight.
- For multi-bit buses, all bits must change simultaneously **or** be Gray coded.

- FLOP_COUNT=2 for relaxed MTBF, 3 for production (default)
- Do **not** use for pulses (sync_pulse) or streaming data (fifo_async)

Gray-coded input must be REGISTERED in the source domain before it crosses. The single-bit-change guarantee is a property of a registered counter sequence, not of the encoding in the abstract. Feeding this module a Gray value straight out of combinational logic re-introduces exactly the hazard Gray coding exists to remove: the bits arrive with different delays, and the synchronizer can latch a transient that was never a real state. Register the Gray value, then cross it. The same applies to a Gray value that can *jump* rather than increment by one – multiple bits change and no encoding saves you.

1.4 cdc_open_loop

Source holds `valid` + data stable for a stretch window long enough that the destination cannot miss it. No acknowledge, so the lowest area and latency of the data-carrying crossings – at the cost of no backpressure.

cdc_open_loop parameters

Parameter	Default	Description
DATA_WIDTH	8	Width of the data bus
STRETCH_CYCLES	8	Source clocks to hold valid+data stable (AUTO_STRETCH=0)
SYNC_STAGES	2	Destination synchronizer depth (2-4)
AUTO_STRETCH	0	0 = use STRETCH_CYCLES; 1 = compute from clock frequencies
SRC_CLK_HZ	25_000_000	Source clock frequency (AUTO_STRETCH=1)
DST_CLK_HZ	100_000_000	Destination clock frequency (AUTO_STRETCH=1)

cdc_open_loop ports

Port	Direction	Width	Description
clk_src / rst_src_n	input	1	Source domain clock / async reset
src_valid	input	1	Single-cycle pulse: data is valid
src_data	input	DATA_WIDT H	Data to transfer
src_busy	output	1	High during stretch countdown
clk_dst / rst_dst_n	input	1	Destination domain clock / async reset
dst_valid	output	1	Single-cycle pulse: data latched
dst_data	output	DATA_WIDT H	Latched data, stable until next dst_valid

The failure mode to watch: sending a new transfer before the previous one is sampled. Without an acknowledge the source cannot know the destination latched the data. Respect src_busy, or space transfers by at least SYNC_STAGES + 1 destination clocks.

1.5 cdc_2_phase_handshake

Toggle (NRZ) closed-loop handshake. Two synchronizer crossings per transfer instead of four.

Read [Reset Considerations](#) before choosing this module. It is pin-compatible with the 4-phase variant but not behaviorally equivalent under independent resets.

cdc_2_phase_handshake parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1+)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts

Parameter	Type	Default	Description
			src_timeout after stall

1.5.1 Shared handshake interface

Both handshake modules expose an identical port list, which is why swapping them is a one-word edit:

Shared handshake interface ports

Port	Direction	Width	Description
clk_src / rst_src_n	input	1	Source domain clock / async reset
src_valid	input	1	Source indicates data valid
src_ready	output	1	Handshake ready (asserted when idle)
src_data	input	DATA_WIDT H	Data from source domain
src_timeo ut	output	1	Stall timeout (when TIMEOUT_CYCLES > 0)
clk_dst / rst_dst_n	input	1	Destination domain clock / async reset
dst_valid	output	1	Data valid to receiver
dst_ready	input	1	Receiver ready
dst_data	output	DATA_WIDT H	Data transferred to destination domain

1.5.2 Theory of operation

Source FSM

S_IDLE src_ready=1. On src_valid: latch data, toggle r_req_tog, drop src_ready, go to S_WAIT_ACK.
S_WAIT_ACK Wait for ack edge (w_ack_event). On edge: src_ready=1, S_IDLE.

Destination FSM

D_IDLE On req edge (w_req_event): copy data into r_dst_data, raise dst_valid, go to D_WAIT_READY.
D_WAIT_READY Hold dst_valid. On dst_ready: drop dst_valid, toggle r_ack_tog, return to D_IDLE.

Edge detection

event = current_sync_output ^ previous_sync_output

1.5.3 Swapping between variants

Consequences of swapping handshake variants

Direction	Consequence
4-phase -> 2-phase	Gains throughput. Silently forfeits independent-reset tolerance. Only safe when both domains share one reset.
2-phase -> 4-phase	Costs ~2 destination clocks of latency. Always safe.

1.6 cdc_4_phase_handshake

Level (RZ) closed-loop handshake: req rises, ack rises, req falls, ack falls. Slower than 2-phase, and tolerant of independent domain resets.

cdc_4_phase_handshake parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1 to 1024+)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts src_timeout after stall
FAST_PATH	bit	0	1 = destination fast-path when dst_ready already high

Ports are identical to the [shared handshake interface](#).

1.6.1 Theory of operation

Source FSM

S_IDLE On src_valid: capture data, r_req_src=1, S_WAIT_ACK.
S_WAIT_ACK On synchronized ack: r_req_src=0, S_WAIT_ACK_CLR.
S_WAIT_ACK_CLR On ack cleared: src_ready=1, S_IDLE.

Destination FSM

```

D_IDLE          On synchronized req: latch data, dst_valid=1,
D_WAIT_READY.
D_WAIT_READY    On dst_ready: r_ack_dst=1, dst_valid=0,
D_WAIT_REQ_CLR.
D_WAIT_REQ_CLR  On req cleared: r_ack_dst=0, D_IDLE.

```

Data stability guarantee: data is captured at REQ assertion and held until ACK completion, so it is stable for the entire crossing regardless of clock ratio.

Latency: ~7-8 destination clocks per transfer (vs ~5-6 for 2-phase).

1.7 Async FIFO Pointers (Gray and Johnson)

For streaming data, synchronize the read and write **pointers** of a dual-port memory rather than the data. Pointers use an encoding where only one bit changes per increment, so any sample during a transition yields either the old or the new value – never a corrupted intermediate. The memory itself needs no CDC.

```

Write Domain:                                Read Domain:
wr_ptr (binary) ---> bin2gray --->          gray2bin ---> compare with
rd_ptr                                         [sync N stages]          for empty
detection

```

```

gray2bin <--- [sync N stages] <--- bin2gray <--- rd_ptr (binary)
compare with wr_ptr for full detection

```

1.7.1 Gray vs Johnson encoding

Both `fifo_async` (rtl/common) and `gaxi_fifo_async` (rtl/amba/gaxi) select the encoding with a `USE_JOHNSON` parameter:

Async FIFO pointer encodings

USE_JOHNSON	Encoding	Pointer width	Converter	Legal DEPTH
0 (default)	Gray	$\log_2(\text{DEPTH}) + 1$	gray2bin (combinational)	power of 2 only
1	Johnson	DEPTH	johnson2bin (registered)	any even depth

An illegal combination (Gray with a non-power-of-2 depth) fails at **elaboration** with an explicit \$error, not at runtime.

1.7.1.1 State walk: why the pointers are different widths

Both encodings guarantee the same CDC-critical property – exactly one bit changes per increment – but they buy it very differently. Walking a 3-bit Gray counter through its full cycle and putting the equivalent Johnson counter beside it shows why.

A 3-bit Gray counter has $2^{*3} = 8$ states. To reach the same 8 states a Johnson counter needs $2N = 8$, i.e. **4 bits** – it only ever produces $2 \times \text{WIDTH}$ states, not 2^{*}WIDTH . The Johnson sequence fills with ones from the LSB, then flushes them back out with zeros; that is the whole counter (counter_johnson.sv:32: {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]}).

The two “which bit” columns give the **bit POSITION that flipped** (0 = LSB). They are not counts. The Hamming-distance columns state the count explicitly, and it is 1 on every row – that is the property being demonstrated.

3-bit Gray vs the equivalent 4-bit Johnson counter, full cycle. “flips bit#” is a bit POSITION; “Ham. dist” is the number of bits that changed, which is 1 everywhere including the wrap.

Step	Gray (3-bit)	flips bit#	Ham. dist	Johnson (4-bit)	flips bit#	Ham. dist	Johnson phase
0	000	0	1	0000	0	1	ones in
1	001	1	1	0001	1	1	ones in
2	011	0	1	0011	2	1	ones in
3	010	2	1	0111	3	1	ones in
4	110	0	1	1111	0	1	zeros in
5	111	1	1	1110	1	1	zeros in
6	101	0	1	1100	2	1	zeros in
7	100	2	1	1000	3	1	zeros in
wrap	-> 000	2	1	-> 0000	3	1	–

Every step – **including the wrap back to state 0** – flips exactly one bit in both encodings. Hamming distance 1 is the only property the synchronizer needs, which is why both are safe to cross.

The Johnson wrap is single-bit, and this is easy to get wrong. A reviewer of this document read the earlier “bit changed” column as a bit *count* and concluded the Johnson wrap 1000 -> 0000 changed four bits. It changes one (bit 3). A twisted-ring counter is specifically constructed so

that the wrap is a single-bit transition like every other step – that is what makes the 2N cycle usable as a CDC pointer at all.

The cost difference is in the width, and it compounds:

Pointer width by encoding – Gray is logarithmic, Johnson is linear

States needed (2 x DEPTH)	DEPT		Johnson pointer bits
	H	Gray pointer bits	
8	4	3	4
16	8	4	8
32	16	5	16
64	32	6	32
128	64	7	64

Gray is $\log_2$ in the state count; Johnson is linear. At DEPTH=4 the difference is one flop; at DEPTH=64 it is 57 flops per pointer, and each pointer is duplicated per domain and again per synchronizer stage. That is the entire argument for Gray being the default – and equally, why Johnson stays viable only at the modest depths where non-power-of-2 sizing actually matters.

What Johnson buys for that width is the freedom to stop at any even count: its 2N progression lands on 6, 10, 14 just as naturally as 8 or 16, whereas Gray's $2^{**}N$ can only land on powers of two.

1.7.2 Sizing the depth

$\text{depth} \geq \text{burst_length} * (1 - \text{read_rate} / \text{write_rate})$

where $\text{rate} = \text{freq} * \text{duty_cycle}$. Round up, then apply the encoding's constraint: Gray rounds to the next power of two; Johnson rounds to the next even number.

This formula assumes the reader keeps draining throughout the burst. It sizes for a *sustained rate mismatch*, not for burst isolation. If the reader can stall completely – arbitration loss, a blocked downstream, a paused consumer – then nothing drains during the stall and you need $\text{depth} \geq \text{burst_length}$ to avoid dropping writes. Between those extremes, size for the worst-case drain rate the reader can guarantee, not its average.

One term the formula above leaves out: the full flag does not track the true fill level instantly. It is derived from a pointer that has been synchronised into the write domain, so it lags real reads by N_FLOP_CROSS write clocks. Production sizing therefore adds N_FLOP_CROSS slots on top of the raw depth – two for the default synchroniser, three if you have configured a third stage. The

worked example below shows the raw depth only, so that it lines up with the storage-overhead tables that follow; add the margin before committing to a number.

Spreadsheet. docs/fifo_depth_calculator_v2.xlsx does this whole calculation – rate-from-duty, raw depth, synchroniser margin, then the Gray power-of-two and Johnson even-number rounding side by side, with the slot and percentage saving between them. It carries worked example cases and a one-page cheat sheet. Note that it deliberately does *not* model clock drift: crystal tolerance is ppm-level and moves the answer by well under one slot over any bounded burst, and a divergence large enough to matter is a steady-state violation whose fix is back-pressure, not a deeper FIFO. To size a worst case, enter the worst-case frequencies directly – fastest writer, slowest reader.

Worked example. Writer at 100 MHz bursts 100 back-to-back writes; reader pulls 1 word/clock at 80 MHz:

Worked depth-sizing example

Step	Value
Burst duration	$100 / 100 \text{ MHz} = 1 \mu\text{s}$
Reads during burst	$1 \mu\text{s} \times 80 \text{ MHz} = 80$
Net build-up	$100 - 80 = \mathbf{20 \text{ words}}$
Gray FIFO (power-of-2)	round up to 32
Johnson FIFO (even)	20 as-is

Gray adds 12 unused slots, and the nominal overhead scales with data width:

Nominal storage overhead of Gray depth rounding by data width

DATA_WIDTH	Gray depth 32	Johnson depth 20	Nominal saving
32 b	1024 b	640 b	384 b (38%)
64 b	2048 b	1280 b	768 b (38%)
256 b	8192 b	5120 b	3072 b (38%)
1024 b	32 768 b	20 480 b	12 288 b (38%)

Those are *nominal* bit counts. Whether they translate into real area depends entirely on how the array is implemented – see the walkthrough below, which is the case most people actually hit.

Add margin for synchronizer latency if you are near the safe depth – the backpressure to the writer lags real fill level by `N_FLOP_CROSS` write clocks.

1.7.3 Walkthrough: is an even-depth FIFO worth it at 512 bits?

This is the question that motivates `USE_JOHNSON`, and the honest answer is “sometimes” – the naive 38% figure above is frequently zero in practice. Work it through.

Scenario. A 512-bit streaming path. Writer at 250 MHz bursts 96 words back-to-back; reader drains 1 word/clock at 200 MHz.

Step 1 – required depth.

$$\text{depth} \geq 96 * (1 - 200/250) = 96 * 0.2 = 19.2 \rightarrow 20 \text{ words}$$

Step 2 – apply the encoding constraint.

512-bit walkthrough: depth after applying the encoding constraint

Encoding	Legal depth	Chosen	Entries wasted
Gray (USE_JOHNSON=0)	power of 2	32	12
Johnson (USE_JOHNSON=1)	any even	20	0

Step 3 – nominal storage delta.

512-bit walkthrough: nominal storage delta

	Depth	Bits
Gray	32 x 512	16 384 b (2 KiB)
Johnson	20 x 512	10 240 b (1.25 KiB)
Delta	12 entries	6 144 b (768 B)

Step 4 – what Johnson costs. The pointer is DEPTH bits wide instead of $\log_2(\text{DEPTH})+1$, and it is duplicated per domain and again per synchronizer stage. With N_FLOP_CROSS=2:

512-bit walkthrough: pointer flop cost

	Gray (depth 32)	Johnson (depth 20)
Pointer width	6 b	20 b
Per domain: own binary + encoded pointer	6 + 6 = 12	6 + 20 = 26
Per domain: synchronized remote pointer	2 x 6 = 12	2 x 20 = 40
Per domain: gray-	0 (combinational)	6 (registered)

	Gray (depth 32)	Johnson (depth 20)
--	-----------------	--------------------

>bin converter

Both domains **~48 flops** **~144 flops**

So Johnson costs roughly **+96 flops** to save **6 144 memory bits**. If the memory is flops, that is a ~64:1 win. If it is not, the trade can invert completely.

Step 5 – the part that decides it: memory granularity. MEM_STYLE (FIFO_AUTO / FIFO_SRL / FIFO_BRAM) determines whether 12 fewer entries costs 12 fewer entries of area:

512-bit walkthrough: whether the saving is real, by memory style

MEM_STYLE	Granularity	Depth 32 vs 20	Verdict
Flop array	1 entry	16 384 vs 10 240 flops	Johnson wins big – saving is exactly proportional, and dwarfs the +96 pointer flops
FIFO_SRL	32 deep per LUT (SRL32)	Both ≤ 32 , so 512 LUTs either way	No saving. Johnson costs +96 flops for nothing
FIFO_BRAM	512 deep x 72 b per BRAM	512-bit width needs $\text{ceil}(512/72) = 8$ BRAMs, and both depths are far under 512	No saving. Same 8 BRAMs either way

Conclusion for this example. At 512 bits and depth ~20, Johnson is a clear win *only* if the array is implemented as flops. Under SRL or BRAM the depth rounding is absorbed by the primitive's own granularity, and choosing Johnson is a pure loss of ~96 flops plus a wider pointer comparator on the critical path.

When even-depth genuinely pays.

1. **Flop-based arrays** – no granularity, saving is proportional. The most common real win, and it grows with DATA_WIDTH.

2. **The depth is externally constrained, not free to round up.** If you need exactly 20 entries to cover a 20-beat burst, a credit limit, or a descriptor count, rounding to 32 is not “spare capacity”, it is 12 entries of dead silicon you provisioned to satisfy an encoding.
3. **Crossing a primitive boundary.** If the rate analysis lands at 33 and Gray forces 64, an SRL32-based FIFO goes from 2 primitives per bit to 2 – no help – but a flop array pays the full 31 entries. Check where your required depth sits relative to the primitive’s step size, not relative to the next power of two.

1.7.3.1 On an ASIC the granularity argument disappears

Everything in Step 5 above is an FPGA argument. SRL32 and BRAM have a fixed step size, and that step size is what absorbs the rounding and erases Johnson’s advantage. On an ASIC there is no such step: both **register files** and **compiled SRAM** are generated to a requested word count, and every memory compiler in common use accepts an even depth. So the depth you compute is the depth you instantiate, and an odd rounding to the next power of two is pure wasted silicon rather than something a primitive was going to charge you for anyway.

Worked case: a required depth of 36. Take the same 512-bit path, but with a rate analysis that lands on 36 entries – above 32, so Gray must round to 64.

Depth-36 case: storage delta when Gray must round 36 up to 64

	Depth	Storage at 512 b
Gray (power of 2)	64	32 768 b (4 KiB)
Johnson (even)	36	18 432 b (2.25 KiB)
Delta	28 entries	14 336 b (1.75 KiB)

Gray pays for 28 entries it will never use – 44 percent of the array – because 36 is not a power of two. The pointer cost runs the other way, by the same accounting as Step 4:

Depth-36 case: pointer flop cost

	Gray (depth 64)	Johnson (depth 36)
Pointer width	7 b	36 b
Per domain: own binary + encoded pointer	$7 + 7 = 14$	$7 + 36 = 43$
Per domain: synchronized remote pointer	$2 \times 7 = 14$	$2 \times 36 = 72$

	Gray (depth 64)	Johnson (depth 36)
Per domain: bin converter	0 (combinational)	7 (registered)
Both domains	~56 flops	~244 flops

Johnson costs about **+188 flops** to save **14 336 memory bits** – roughly a 76:1 return, better than the depth-20 case because the rounding gap is wider.

Which ASIC memory style. At depth 36 both are available, and the choice is about aspect ratio rather than about the encoding:

Depth-36 case: ASIC memory styles, all of which accept an even depth

Style	Even depth?	Fit at 36 x 512
Flop array	Yes, no granularity at all	Works, but 18 432 flops is a lot of area and a synthesis burden
Register file	Yes, word count is a compiler argument	Good fit. Shallow and wide is what register files are for
Compiled SRAM	Yes, word count is a compiler argument	Works, but 36 words is shallow for SRAM; check the compiler's minimum depth and its area per bit at this aspect ratio

The practical answer at this shape is usually a register file: it takes depth 36 directly, it is denser than a flop array, and it avoids the periphery overhead that makes a 36-word SRAM inefficient. Confirm against your own compiler – minimum depth, word-count granularity (some compilers step in multiples of 4 or 8 words, which would reintroduce a smaller version of the FPGA problem), and area per bit at a shallow, wide aspect ratio.

Rule of thumb. Compute the required depth first. Then ask what the memory maps to and what its step size is. Only if the rounding actually crosses a step boundary – or there is no step, as with flops, register files, and compiled SRAM – does `USE_JOHNSON=1` buy you anything. On an ASIC that condition is usually met, so Johnson is worth pricing whenever the required depth is not already a power of two. On an FPGA it usually is not met, and Gray is the default: narrower pointers, a combinational `gray2bin` instead of a registered `johnson2bin`, and one less thing to reason about.

1.8 SDC constraints

Both handshakes need the control toggles constrained as bounded crossings, and the quasi-static data bus constrained by `set_max_delay`, **not** `set_false_path`:

```
# 1. Req / Ack single-bit crossings
set_max_delay -datapath_only \
  -from [get_pins u_cdc/r_req_tog_reg/C] \
  -to   [get_pins u_cdc/r_req_sync_reg[0]/D] \
  <dst_period_ns>

set_max_delay -datapath_only \
  -from [get_pins u_cdc/r_ack_tog_reg/C] \
  -to   [get_pins u_cdc/r_ack_sync_reg[0]/D] \
  <src_period_ns>

# 2. Data bus (quasi-static, protected by the handshake)
set_max_delay -datapath_only \
  -from [get_pins u_cdc/r_src_data_hold_reg[*]/C] \
  -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
  <dst_period_ns>
```

Why -datapath_only, and why not set_false_path:

- set_false_path tells the tool to ignore the path entirely. The data bus is then free to take arbitrarily long, but the handshake only holds it stable for a bounded window – so the destination can latch a value that has partially updated. The crossing is unconstrained, not safe.
- set_max_delay **without** -datapath_only still includes clock skew and clock-uncertainty terms in the calculation. Across asynchronous domains those terms are meaningless – the tool has no defined phase relationship to work from – so you get pessimistic, unfixable violations that tempt people back to set_false_path.
- set_max_delay -datapath_only constrains **only the combinational datapath delay** from source register to destination register, which is exactly the quantity that must fit inside the handshake’s stable window. This is the one that expresses the real requirement.

Set the value to the destination clock period (or the source period, for the ack/return direction) unless you have a specific reason to tighten it.

1.9 Common mistakes

1. Multi-flop synchronizer on a multi-bit bus that changes simultaneously. Safe only for single-bit or Gray-coded values. If multiple bits change at once, different bits may be sampled from different time points, producing a value that never existed. Fix: handshake or async FIFO.

2. **set_false_path on CDC data buses.** It tells the tool to ignore timing entirely, but the data bus must arrive within a bounded window. Use `set_max_delay -datapath_only`.
 3. **Assuming async FIFO depth 2 is enough.** Depth 2 has one usable entry, and synchronized pointers lag by `SYNC_STAGES` cycles. Use `depth >= 4`.
 4. **Open-loop transfer before the previous one is sampled.** Without an ack the first transfer is silently lost. Minimum spacing `SYNC_STAGES + 1` destination clocks.
 5. **Choosing 2-phase for speed without checking the reset domains.** See [Reset Considerations](#). This one has cost real silicon debug time.
-

1.10 Verification status

The formal harness at `formal/amba/cdc_handshake/formal_cdc_handshake.sv` wires the DUT as:

```
.clk_src (clk), .rst_src_n (rst_n),  
.clk_dst (clk), .rst_dst_n (rst_n),
```

Single clock, single reset for both domains. The proof therefore **cannot express asymmetric reset or asynchronous clocks**. Treat “formally verified” for the handshakes as scoped to protocol correctness under a common clock and reset.

The reset behavior described in this document is argued from the RTL encoding and – for the 2-phase hazard – confirmed on silicon. It is **not** currently covered by the formal proof or by a directed test. Extending the harness to independent clocks and resets would cover it, and would be expected to fail on the 2-phase variant.

1.11 Module reference

CDC module reference

Module	RTL	Filelist	Test
cdc_synchronizer	rtl/amba/ cdc/ cdc_synchr onizer.sv	cdc_synchronizer.f	–
cdc_open_loop	rtl/amba/ cdc/ cdc_open_l oop.sv	cdc_open_loop.f	val/amba/ test_cdc_ope n_loop.py
cdc_2_phase_hands	rtl/amba/	cdc_2_phase_handshake.	val/amba/

Module	RTL	Filelist	Test
hake	cdc/ cdc_2_phas e_handshak e.sv	f	test_cdc_2_p hase_handsha ke.py
cdc_4_phase_hands hake	rtl/amba/ cdc/ cdc_4_phas e_handshak e.sv	cdc_4_phase_handshake. f	val/amba/ test_cdc_4_p hase_handsha ke.py
fifo_async	rtl/ common/ fifo_async .sv	rtl/common/filelists/ fifo_async.f	val/common/ test_fifo_bu ffer_async.p y
gaxi_fifo_async	rtl/amba/ gaxi/ gaxi_fifo_ async.sv	—	—

Protocol-level CDC built on these blocks: apb_slave_cdc, apb_slave_cdc_cg, apb5_slave_cdc, apb5_slave_cdc_cg, gaxi_skid_buffer_async, axi4_to_apb_shim.

Supporting primitives (rtl/common): glitch_free_n_dff_arn, bin2gray, gray2bin, johnson2bin, counter_bingray, counter_johnson, reset_sync, sync_pulse.

1.12 Navigation

- [Back to CDC Index](#)
- [Back to RTLAmba Index](#)

2 Glitch-Free N-DFF Synchronizer (glitch_free_n_dff_arn.sv)

2.1 Purpose

Provides a parameterized multi-stage synchronizer for safe clock domain crossing (CDC). Reduces metastability risk through configurable stages of flip-flops while maintaining data integrity across asynchronous clock domains.

2.2 Ports

2.2.1 Input Ports

- **clk** - Destination domain clock
- **rst_n** - Destination domain active-low reset
- **d[WIDTH-1:0]** - Input data from source clock domain

2.2.2 Output Ports

- **q[WIDTH-1:0]** - Synchronized output data in destination domain

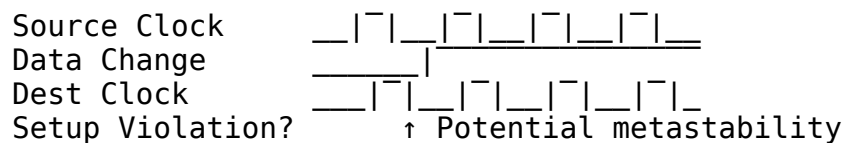
2.2.3 Parameters

- **FLOP_COUNT** - Number of synchronizer flip-flop stages (default: 3)
- **WIDTH** - Data width in bits (default: 4)

2.3 Clock Domain Crossing Theory

2.3.1 Metastability Problem

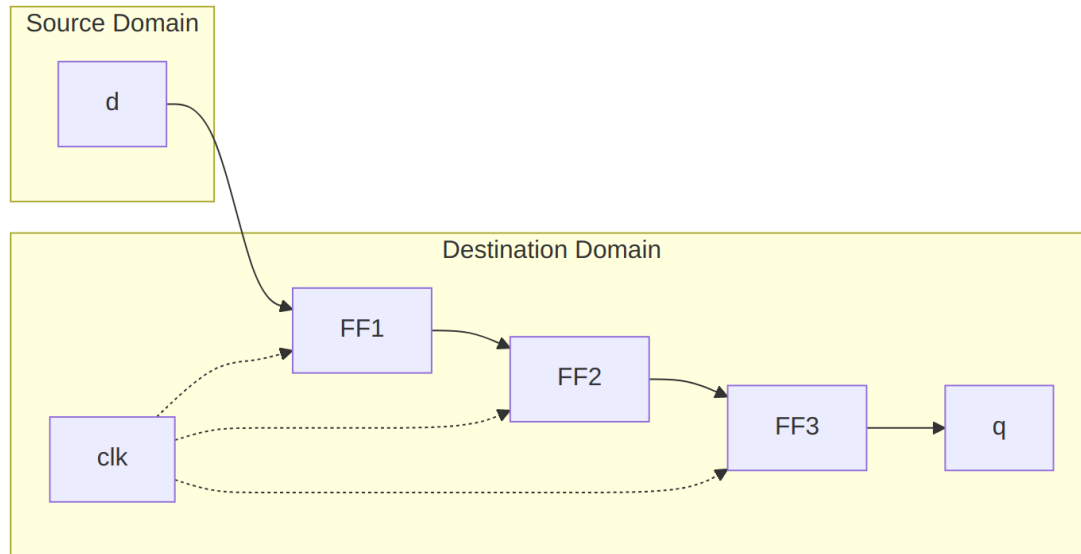
When signals cross between asynchronous clock domains:



2.3.2 Metastability Effects

- **Undefined output:** Neither 0 nor 1, but intermediate voltage
- **Extended resolution:** Takes longer than normal clock period
- **Propagation:** Can corrupt downstream logic
- **System failure:** Can cause entire design malfunction

2.4 Multi-Stage Synchronizer Solution



Architecture Overview

2.4.1 Stage-by-Stage Analysis

1. **Stage 1 (FF1):** Captures source data, may go metastable
2. **Stage 2 (FF2):** Allows metastability to resolve, low probability of propagation
3. **Stage 3+ (FFN):** Further reduces MTBF (Mean Time Between Failures)

2.5 Implementation Details

2.5.1 Packed Array Structure

logic [FC-1:0][WIDTH-1:0] r_q_array;

- **Efficient storage:** Single declaration for all stages
- **Clear indexing:** r_q_array[0] is first stage, r_q_array[FC-1] is last
- **Synthesis friendly:** Maps well to FPGA/ASIC resources

2.5.2 Shift Register Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Reset all flip-flops to zero
        r_q_array <= {FC{{DW{1'b0}}}};
    end else begin
        // Load new data into first stage
```

```

    r_q_array[0] <= d;
    // Shift existing data through remaining stages
    for (int i = 1; i < FC; i++) begin
        r_q_array[i] <= r_q_array[i-1];
    end
end
end

```

2.5.3 Reset Behavior

- **Synchronous reset:** Uses destination domain reset
- **Complete reset:** All stages reset to zero
- **Nested replication:** {FC{{DW{1'b0}}}} creates FC copies of DW zeros

2.6 MTBF (Mean Time Between Failures) Analysis

2.6.1 MTBF Calculation

$$\text{MTBF} = (e^{(t_{\text{res}}/\tau)}) / (T_0 \times f_{\text{clk}} \times f_{\text{data}})$$

Where:

- t_{res} = Resolution time available (clock period - setup - routing)
- τ = Flip-flop metastability resolution time constant (~200ps typical)
- T_0 = Metastability aperture / window constant (device-specific, ~1e-12 s)
- f_{clk} = Destination clock frequency
- f_{data} = Data transition frequency

τ appears **ONLY in the exponent**. The denominator constant is T_0 , not τ – they are different device parameters and are not interchangeable. Both are process/library specific: take them from the vendor's metastability characterization data, not from a datasheet timing table.

The stage-count table below is a **rule of thumb** for relative improvement, not a result derived from this formula. Use the formula only with real τ/T_0 values for your target device; otherwise rely on the table for sizing.

2.6.2 Stage Count Impact

Stages	Relative MTBF	Typical MTBF
1	1× (baseline)	Minutes
2	~1000×	Hours
3	~1,000,000×	Years
4	~1,000,000,000×	Millennia

2.6.3 Practical Guidelines

- **2 stages:** Minimum for most applications
- **3 stages:** Recommended for high-reliability systems
- **4+ stages:** Extreme reliability requirements (aerospace, medical)

2.7 Latency vs. Reliability Trade-off

2.7.1 Synchronizer Latency

- **Minimum latency:** 2 clock cycles (2-stage synchronizer)
- **General latency:** FLOP_COUNT clock cycles
- **Fixed delay:** Independent of data pattern

2.7.2 Performance Impact

```
// 2-stage: 2 cycle latency, good MTBF  
glitch_free_n_dff_arn #(.FLOP_COUNT(2)) fast_sync (...);
```

```
// 3-stage: 3 cycle latency, excellent MTBF  
glitch_free_n_dff_arn #(.FLOP_COUNT(3)) safe_sync (...);
```

```
// 4-stage: 4 cycle latency, extreme MTBF  
glitch_free_n_dff_arn #(.FLOP_COUNT(4)) ultra_safe_sync (...);
```

2.8 Usage Guidelines

2.8.1 Appropriate Use Cases

- **Control signals:** Reset, enable, mode switches
- **Slow status signals:** Error flags, configuration bits
- **Gray code pointers:** FIFO read/write pointers
- **Single-bit signals:** Generally safe for multi-bit if Gray coded

2.8.2 Inappropriate Use Cases

- **High-speed data buses:** Use proper CDC techniques (handshaking, FIFOs)
- **Binary counters:** Multiple bits can transition simultaneously
- **Address buses:** Binary addresses are not CDC-safe
- **Clock signals:** Never synchronize clocks this way

2.8.3 Multi-Bit Considerations

```
// SAFE: Gray-coded FIFO pointers
glitch_free_n_dff_arn #(.WIDTH(5)) gray_sync (
    .d(gray_pointer),           // Gray code - only 1 bit changes
    .q(sync_gray_pointer)
);

// UNSAFE: Binary counter synchronization
// Don't do this - multiple bits can change simultaneously
glitch_free_n_dff_arn #(.WIDTH(8)) bad_sync (
    .d(binary_counter),        // Binary - multiple bit transitions
    .q(sync_counter)           // May capture inconsistent values
);
```

2.9 Advanced Applications

2.9.1 Reset Synchronizer Pattern

```
// Asynchronous assert, synchronous deassert
logic reset_sync;
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(1)) reset_sync_inst (
    .clk(dest_clk),
    .rst_n(1'b1),           // No reset feedback
    .d(~async_reset),       // Invert for active-low
    .q(reset_sync)
);
```

```
assign sync_reset_n = reset_sync & ~async_reset;
```

2.9.2 Enable Signal Synchronization

```
glitch_free_n_dff_arn #(.FLOP_COUNT(3), .WIDTH(1)) enable_sync (
    .clk(fast_clk),
    .rst_n(rst_n),
    .d(slow_enable),
    .q(fast_enable_sync)
);
```

2.9.3 Status Flag Collection

```
// Synchronize multiple independent status signals
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(4)) status_sync (
    .clk(dest_clk),
    .rst_n(rst_n),
    .d({error_flag, done_flag, busy_flag, ready_flag}),
    .q({sync_error, sync_done, sync_busy, sync_ready})
);
```

2.10 Synthesis Considerations

2.10.1 FPGA Implementation

- **Dedicated resources:** Maps to flip-flop primitives
- **Placement:** Tools may place stages in different locations
- **Timing constraints:** Require proper CDC constraints

2.10.2 Timing Constraints

```
# Constrain the synchronizer input as asynchronous
set_false_path -to [get_pins sync_inst/r_q_array[0]/D]

# Or set maximum delay if there's some relationship
set_max_delay -to [get_pins sync_inst/r_q_array[0]/D] 2.0
```

2.10.3 Optimization Prevention

```
// Synthesis attributes to prevent optimization
(* ASYNC_REG = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
(* DONT_TOUCH = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
```

2.11 Verification Strategies

2.11.1 Functional Verification

- **Data integrity:** Verify output eventually matches input
- **Reset behavior:** Check proper reset operation
- **Latency verification:** Confirm expected delay

2.11.2 Metastability Testing

```
// Stress test with rapid data changes
initial begin
    forever begin
        @(posedge source_clk);
        d <= $random();
        #($random() % 100); // Random timing
    end
end
```

2.11.3 Formal Verification

- **Eventually properties:** Data will eventually propagate
- **Reset properties:** Reset behavior verification
- **No X propagation:** Ensure no unknown values propagate

2.12 Debug Features

2.12.1 Waveform Analysis

```
// Flatten array for waveform viewing
wire [(DW*FC)-1:0] flat_r_q;
genvar i;
generate
    for (i = 0; i < FC; i++) begin : gen_flatten_memory
        assign flat_r_q[i*DW+:DW] = r_q_array[i];
    end
endgenerate
```

2.12.2 Simulation Visibility

- **All stages visible:** Can observe data propagation through stages
- **Metastability injection:** Can inject X values for testing
- **Timing analysis:** Observe setup/hold violations

2.13 Related Modules

- **FIFO CDC modules:** Use this for pointer synchronization
- **Reset synchronizers:** Specialized CDC for reset signals
- **Handshake synchronizers:** For complex data CDC
- **Pulse synchronizers:** For single-cycle pulse CDC

2.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

3 Binary to Gray Code Converter

3.1 Overview

The bin2gray module implements a purely combinational binary-to-Gray code converter. Gray code (also known as reflected binary code or unit distance code) is a binary numeral system where two successive values differ in only one bit. This property makes Gray code essential for reducing glitches and metastability in digital systems, particularly in asynchronous designs and clock domain crossings.

3.2 Module Declaration

```
module bin2gray #(
    parameter int WIDTH = 4
) (
    input wire [WIDTH-1:0] binary,
    output wire [WIDTH-1:0] gray
);
```

3.3 Parameters

3.3.1 WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Bit width of both input and output
- **Range:** Any positive integer ≥ 1
- **Common Values:** 4, 8, 16, 32 for address counters and data buses
- **Impact:** Determines the number of XOR gates required

3.4 Ports

3.4.1 Inputs

Port	Width	Type	Description
binary	WIDTH	wire	Input binary value

3.4.2 Outputs

Port	Width	Type	Description
gray	WIDTH	wire	Output Gray code value

3.5 Gray Code Theory

3.5.1 Mathematical Definition

Gray code conversion follows a simple mathematical relationship: - **MSB:** $\text{gray}[\text{WIDTH}-1] = \text{binary}[\text{WIDTH}-1]$ (MSB unchanged) - **Other bits:** $\text{gray}[i] = \text{binary}[i] \oplus \text{binary}[i+1]$ for $i = 0$ to $\text{WIDTH}-2$

3.5.2 Why Gray Code?

Gray code solves several problems in digital systems:

1. **Single Bit Transitions:** Adjacent values differ by exactly one bit
2. **Glitch Reduction:** Eliminates intermediate states during transitions
3. **Metastability Prevention:** Safer for asynchronous clock domain crossings
4. **Mechanical Encoders:** Natural for optical/magnetic position encoders

3.5.3 Gray Code Sequence Examples

3.5.3.1 2-bit Gray Code

Decimal	Binary	Gray	Transition
0	00	00	-
1	01	01	bit 0
2	10	11	bit 1
3	11	10	bit 0

3.5.3.2 3-bit Gray Code

Decimal	Binary	Gray	Changed Bit
0	000	000	-
1	001	001	0
2	010	011	1
3	011	010	0
4	100	110	2
5	101	111	0
6	110	101	1
7	111	100	0

3.5.3.3 4-bit Gray Code (Complete Table)

Dec	Binary	Gray	Dec	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010

Dec	Binary	Gray	Dec	Binary	Gray
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

3.6 Implementation Analysis

3.6.1 Core Logic

```

genvar i;
generate
    for (i = 0; i < WIDTH - 1; i++) begin : gen_gray
        assign gray[i] = binary[i] ^ binary[i+1];
    end
endgenerate

```

```

assign gray[WIDTH-1] = binary[WIDTH-1];

```

3.6.2 Logic Structure

The implementation creates a chain of XOR gates: - **Lower bits**: Each gray bit is XOR of current and next binary bits - **MSB**: Directly assigned from binary MSB - **Propagation**: Single level of logic (all XORs in parallel)

3.6.3 Gate-Level Implementation

For WIDTH = 4:

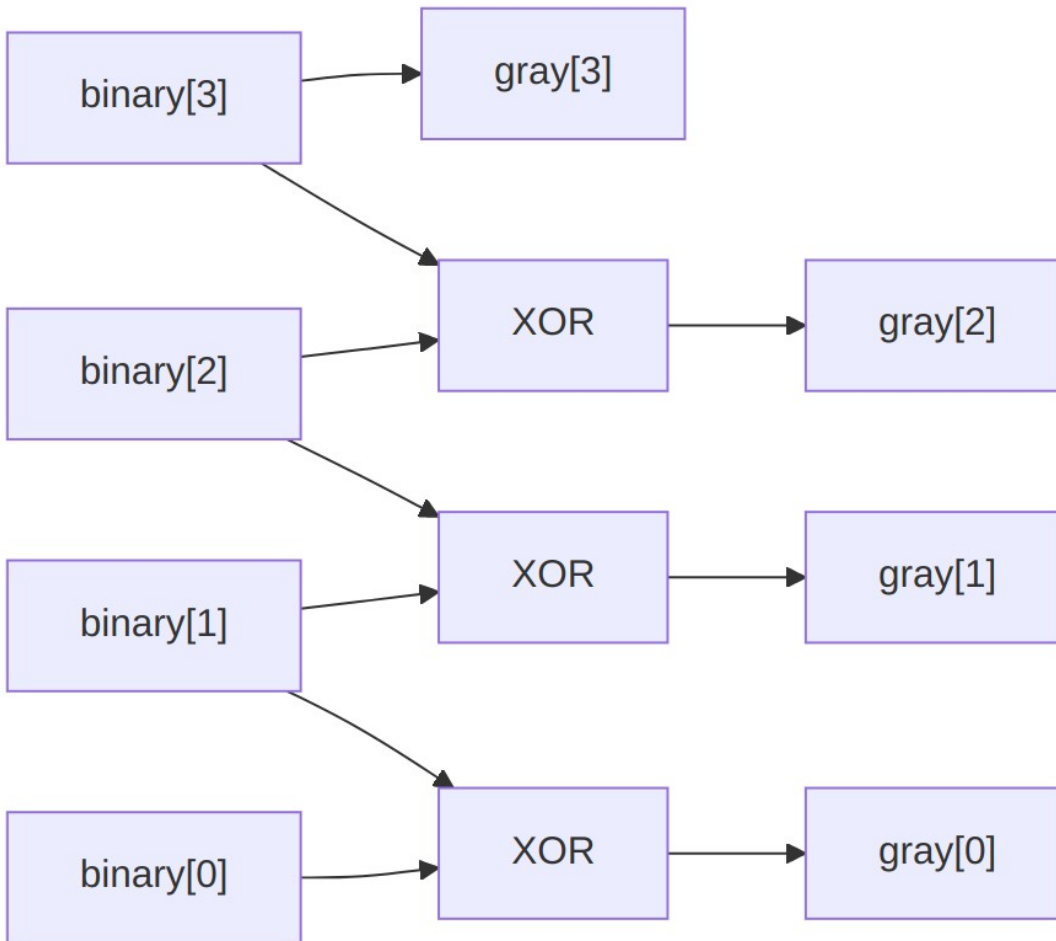


Diagram 2

3.6.4 Timing Characteristics

- **Propagation Delay:** Single XOR gate delay (typically 0.1-0.3ns in modern processes)
- **Critical Path:** Through one XOR gate only
- **Fan-out:** Each binary bit drives at most 2 XOR gates
- **Scalability:** Constant delay regardless of WIDTH

3.7 Design Examples and Applications

3.7.1 1. Asynchronous FIFO Pointer

```

module async_fifo_ptr #(
    parameter int ADDR_WIDTH = 4
) (

```

```

    input  logic          clk,
    input  logic          rst_n,
    input  logic          enable,
    output logic [ADDR_WIDTH:0] gray_ptr,
    output logic [ADDR_WIDTH-1:0] addr
);

    logic [ADDR_WIDTH:0] binary_ptr, binary_ptr_next;

    // Binary counter
    assign binary_ptr_next = enable ? (binary_ptr + 1) : binary_ptr;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            binary_ptr <= 'b0;
        else
            binary_ptr <= binary_ptr_next;
    end

    // Binary to Gray conversion
    bin2gray #(
        .WIDTH(ADDR_WIDTH + 1)
    ) ptr_converter (
        .binary(binary_ptr),
        .gray(gray_ptr)
    );

    // Extract address (lower bits of binary counter)
    assign addr = binary_ptr[ADDR_WIDTH-1:0];

endmodule

```

3.7.2 2. Clock Domain Crossing Counter

```

module cross_domain_counter #(
    parameter int WIDTH = 8
) (
    // Source domain
    input  logic          src_clk,
    input  logic          src_rst_n,
    input  logic          src_enable,

    // Destination domain
    input  logic          dst_clk,
    input  logic          dst_rst_n,
    output logic [WIDTH-1:0] dst_count_binary,
    output logic [WIDTH-1:0] dst_count_gray
);

```

```

// Source domain counter
logic [WIDTH-1:0] src_binary, src_gray;

always_ff @(posedge src_clk or negedge src_rst_n) begin
    if (!src_rst_n)
        src_binary <= 'b0;
    else if (src_enable)
        src_binary <= src_binary + 1;
end

// Convert to Gray in source domain
bin2gray #(.WIDTH(WIDTH)) src_converter (
    .binary(src_binary),
    .gray(src_gray)
);

// Synchronize Gray code to destination domain
logic [WIDTH-1:0] dst_gray_sync;

synchronizer #(.WIDTH(WIDTH)) gray_sync (
    .clk(dst_clk),
    .rst_n(dst_rst_n),
    .data_in(src_gray),
    .data_out(dst_gray_sync)
);

// Convert back to binary in destination domain
gray2bin #(.WIDTH(WIDTH)) dst_converter (
    .gray(dst_gray_sync),
    .binary(dst_count_binary)
);

assign dst_count_gray = dst_gray_sync;

endmodule

```

3.7.3 3. Rotary Encoder Interface

```

module rotary_encoder_interface #(
    parameter int POSITION_WIDTH = 12
) (
    input logic                clk,
    input logic                rst_n,
    input logic [POSITION_WIDTH-1:0] encoder_binary,
    output logic [POSITION_WIDTH-1:0] encoder_gray,
    output logic [POSITION_WIDTH-1:0] position_filtered,
    output logic                position_changed

```

```

);

// Convert encoder binary position to Gray
bin2gray #(
    .WIDTH(POSITION_WIDTH)
) encoder_conv (
    .binary(encoder_binary),
    .gray(encoder_gray)
);

// Synchronize and filter
logic [POSITION_WIDTH-1:0] gray_sync, gray_prev;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        gray_sync <= 'b0;
        gray_prev <= 'b0;
    end else begin
        gray_sync <= encoder_gray;
        gray_prev <= gray_sync;
    end
end

// Detect changes (only one bit should change in Gray code)
logic valid_transition;
assign valid_transition = ($countones(gray_sync ^ gray_prev) <=
1);

// Convert back to binary for position output
gray2bin #(.WIDTH(POSITION_WIDTH)) pos_converter (
    .gray(gray_sync),
    .binary(position_filtered)
);

assign position_changed = valid_transition && (gray_sync !=
gray_prev);

endmodule

```

3.7.4 4. Address Scrambling for Memory Testing

```

module memory_address_scrambler #(
    parameter int ADDR_WIDTH = 16
) (
    input logic [ADDR_WIDTH-1:0] linear_addr,
    output logic [ADDR_WIDTH-1:0] scrambled_addr
);

```

```

logic [ADDR_WIDTH-1:0] gray_addr;

// Convert linear address to Gray code for scrambling
bin2gray #(.WIDTH(ADDR_WIDTH)) scrambler (
    .binary(linear_addr),
    .gray(gray_addr)
);

// Additional scrambling (optional)
assign scrambled_addr = {gray_addr[0], gray_addr[ADDR_WIDTH-1:1]};

endmodule

```

3.7.5 5. Multi-Bit Synchronizer with Gray Code

Prefer the library module. The example below is written out in full to show the mechanism, but production code should instantiate `glitch_free_n_dff_arn` (rtl/common) or `cdc_synchronizer` (rtl/amba/cdc) rather than hand-rolling the flop chain.

Two conditions this example depends on, both easy to get wrong: -
`src_gray` must be **registered in the source domain** before it crosses.
The single-bit-change guarantee belongs to a registered counter sequence, not to the encoding in the abstract; a Gray value straight out of combinational logic can present a transient that was never a real state. -
The source value must **increment by one**. A Gray value that jumps changes multiple bits and no encoding makes that safe.

```

module gray_code_synchronizer #(
    parameter int WIDTH = 8,
    parameter int SYNC_STAGES = 2
) (
    input logic          src_clk,
    input logic          src_rst_n,
    input logic [WIDTH-1:0] src_data,

    input logic          dst_clk,
    input logic          dst_rst_n,
    output logic [WIDTH-1:0] dst_data
);

// Convert to Gray in source domain
logic [WIDTH-1:0] src_gray;

bin2gray #(.WIDTH(WIDTH)) src_conv (
    .binary(src_data),
    .gray(src_gray)

```

```

);

// Multi-stage synchronizer for Gray code
logic [WIDTH-1:0] sync_regs [SYNC_STAGES];

always_ff @(posedge dst_clk or negedge dst_rst_n) begin
    if (!dst_rst_n) begin
        for (int i = 0; i < SYNC_STAGES; i++) begin
            sync_regs[i] <= 'b0;
        end
    end else begin
        sync_regs[0] <= src_gray;
        for (int i = 1; i < SYNC_STAGES; i++) begin
            sync_regs[i] <= sync_regs[i-1];
        end
    end
end

// Convert back to binary in destination domain
gray2bin #(.WIDTH(WIDTH)) dst_conv (
    .gray(sync_regs[SYNC_STAGES-1]),
    .binary(dst_data)
);

endmodule

```

3.8 Companion Gray-to-Binary Converter

3.8.1 Implementation

```

module gray2bin #(
    parameter int WIDTH = 4
) (
    input wire [WIDTH-1:0] gray,
    output wire [WIDTH-1:0] binary
);

    genvar i;

    // MSB is unchanged
    assign binary[WIDTH-1] = gray[WIDTH-1];

    // Other bits: XOR accumulation from MSB down
    generate
        for (i = WIDTH-2; i >= 0; i--) begin : gen_binary
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate

```

endmodule

3.8.2 Gray-to-Binary Truth Table (4-bit)

Gray	Binary	Conversion Process
0000	0000	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 0=0, bin[0]=0 $\oplus$ 0=0
0001	0001	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 0=0, bin[0]=0 $\oplus$ 1=1
0011	0010	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 1=1, bin[0]=1 $\oplus$ 1=0
0010	0011	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 1=1, bin[0]=1 $\oplus$ 0=1

3.9 Advanced Implementations

3.9.1 1. Parameterized Converter with Validation

```
module bin2gray_validated #(
    parameter int WIDTH = 4,
    parameter bit ENABLE_CHECKS = 1
) (
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray,
    output logic valid
);

    // Basic conversion
    bin2gray #(.WIDTH(WIDTH)) converter (
        .binary(binary),
        .gray(gray)
    );
```

```

// Optional validation
generate
    if (ENABLE_CHECKS) begin : validation
        logic [WIDTH-1:0] binary_check;

        // Round-trip conversion check
        gray2bin #(.WIDTH(WIDTH)) check_converter (
            .gray(gray),
            .binary(binary_check)
        );

        assign valid = (binary == binary_check);
    end else begin : no_validation
        assign valid = 1'b1;
    end
endgenerate

endmodule

```

3.9.2 2. Pipelined Converter for High Speed

```

module bin2gray_pipelined #(
    parameter int WIDTH = 32,
    parameter int PIPELINE_STAGES = 2
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] binary_in,
    input logic valid_in,
    output logic [WIDTH-1:0] gray_out,
    output logic valid_out
);

// Pipeline registers
logic [WIDTH-1:0] binary_pipe [PIPELINE_STAGES];
logic valid_pipe [PIPELINE_STAGES];
logic [WIDTH-1:0] gray_pipe [PIPELINE_STAGES];

// Stage 0: Input registration
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        binary_pipe[0] <= 'b0;
        valid_pipe[0] <= 1'b0;
    end else begin
        binary_pipe[0] <= binary_in;
        valid_pipe[0] <= valid_in;
    end
end
end

```

```

// Conversion in stage 0
bin2gray #(.WIDTH(WIDTH)) stage0_conv (
    .binary(binary_pipe[0]),
    .gray(gray_pipe[0])
);

// Additional pipeline stages
generate
    for (genvar s = 1; s < PIPELINE_STAGES; s++) begin :
pipeline_stages
        always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n) begin
                gray_pipe[s] <= 'b0;
                valid_pipe[s] <= 1'b0;
            end else begin
                gray_pipe[s] <= gray_pipe[s-1];
                valid_pipe[s] <= valid_pipe[s-1];
            end
        end
    end
endgenerate

// Output assignment
assign gray_out = gray_pipe[PIPELINE_STAGES-1];
assign valid_out = valid_pipe[PIPELINE_STAGES-1];

endmodule

```

3.9.3 3. Bidirectional Converter

```

module bidirectional_gray_converter #(
    parameter int WIDTH = 8
) (
    input logic direction, // 0: bin→gray, 1: gray→bin
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

    logic [WIDTH-1:0] bin_to_gray_out, gray_to_bin_out;

    // Both converters always active
    bin2gray #(.WIDTH(WIDTH)) b2g (
        .binary(data_in),
        .gray(bin_to_gray_out)
    );

    gray2bin #(.WIDTH(WIDTH)) g2b (

```

```

        .gray(data_in),
        .binary(gray_to_bin_out)
    );

    // Mux output based on direction
    assign data_out = direction ? gray_to_bin_out : bin_to_gray_out;

endmodule

```

3.10 Verification and Testing

3.10.1 Comprehensive Test Bench

```

module tb_bin2gray;

    parameter int WIDTH = 4;
    parameter int MAX_VAL = (1 << WIDTH) - 1;

    logic [WIDTH-1:0] binary, gray;
    logic [WIDTH-1:0] expected_gray;
    logic [WIDTH-1:0] binary_check;

    // DUT
    bin2gray #(.WIDTH(WIDTH)) dut (
        .binary(binary),
        .gray(gray)
    );

    // Reference converter for checking
    gray2bin #(.WIDTH(WIDTH)) check_conv (
        .gray(gray),
        .binary(binary_check)
    );

    // Test sequence
    initial begin
        $display("Testing %d-bit Binary to Gray converter", WIDTH);

        // Test all possible values
        for (int i = 0; i <= MAX_VAL; i++) begin
            binary = i;
            expected_gray = compute_gray_reference(i);

            #1; // Allow propagation

            // Check conversion correctness
            if (gray !== expected_gray) begin
                $error("Mismatch: binary=%b, expected_gray=%b,

```

```

actual_gray=%b",
                                binary, expected_gray, gray);
    end

    // Check round-trip conversion
    if (binary_check != binary) begin
        $error("Round-trip failed: original=%b, recovered=%b",
               binary, binary_check);
    end

    // Check single-bit change property
    if (i > 0) begin
        logic [WIDTH-1:0] prev_gray =
compute_gray_reference(i-1);
        int bit_changes = $countones(gray ^ prev_gray);
        if (bit_changes != 1) begin
            $error("Multiple bit change: %d→%d, gray %b→%b,
changes=%d",
                                i-1, i, prev_gray, gray, bit_changes);
        end
    end

    $display("PASS: %d → binary:%b → gray:%b", i, binary,
gray);
end

    $display("All tests passed!");
    $finish;
end

// Reference Gray code computation
function [WIDTH-1:0] compute_gray_reference(input [WIDTH-1:0]
bin);
    compute_gray_reference[WIDTH-1] = bin[WIDTH-1];
    for (int i = WIDTH-2; i >= 0; i--) begin
        compute_gray_reference[i] = bin[i] ^ bin[i+1];
    end
endfunction
endmodule

```

3.10.2 Property-Based Verification

```

module bin2gray_properties #(
    parameter int WIDTH = 4
);

    logic [WIDTH-1:0] binary, gray;

```

```

// Bind to DUT
bind bin2gray bin2gray_properties #(WIDTH) props_inst (.*);

// Property: MSB preservation
property msb_unchanged;
    gray[WIDTH-1] == binary[WIDTH-1];
endproperty

// Property: Single bit change between consecutive values
property single_bit_change;
    logic [WIDTH-1:0] bin_plus_one = binary + 1;
    logic [WIDTH-1:0] gray_plus_one;

    // Compute Gray code for binary+1
    assign gray_plus_one[WIDTH-1] = bin_plus_one[WIDTH-1];
    for (genvar i = 0; i < WIDTH-1; i++) begin
        assign gray_plus_one[i] = bin_plus_one[i] ^
bin_plus_one[i+1];
    end

    // Check single bit difference
    (binary < (2**WIDTH - 1)) |-> ($countones(gray ^
gray_plus_one) == 1);
endproperty

// Property: Round-trip conversion
property round_trip_conversion;
    logic [WIDTH-1:0] recovered_binary;

    // Convert back to binary
    assign recovered_binary[WIDTH-1] = gray[WIDTH-1];
    for (genvar i = WIDTH-2; i >= 0; i--) begin
        assign recovered_binary[i] = recovered_binary[i+1] ^
gray[i];
    end

    recovered_binary == binary;
endproperty

// Assertions
assert property (msb_unchanged);
assert property (single_bit_change);
assert property (round_trip_conversion);

endmodule

```

3.10.3 Coverage Model

```
covergroup bin2gray_cg;

  cp_binary_values: coverpoint binary {
    bins zero = {0};
    bins powers_of_two[] = {1, 2, 4, 8, 16}; // For appropriate
WIDTH
    bins max_value = {2**WIDTH - 1};
    bins mid_range[] = {[1:2**(WIDTH-1)-1]};
    bins upper_range[] = {[2**(WIDTH-1):2**WIDTH-2]};
  }

  cp_gray_values: coverpoint gray {
    bins all_values[] = {[0:2**WIDTH-1]};
  }

  cp_bit_patterns: coverpoint binary {
    bins alternating_01 = {8'b01010101}; // For WIDTH=8
    bins alternating_10 = {8'b10101010};
    bins all_ones = {'1'};
    bins all_zeros = {'0'};
  }

  // Cross coverage between input patterns and outputs
  cross_binary_gray: cross cp_binary_values, cp_gray_values;

endcovergroup
```

3.11 Synthesis Optimization

3.11.1 Resource Utilization

For different widths, typical FPGA resource usage:

WIDTH	LUTs	Delay (ns)	Max Freq (MHz)
4	3	0.2	800+
8	7	0.2	800+
16	15	0.3	600+
32	31	0.4	500+

3.11.2 Timing Optimization

```
// For critical timing paths, add pipeline register
module bin2gray_registered #(
  parameter int WIDTH = 16
```

```

) (
    input logic          clk,
    input logic          rst_n,
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic [WIDTH-1:0] gray_comb;

    // Combinational conversion
    bin2gray #(.WIDTH(WIDTH)) conv (
        .binary(binary),
        .gray(gray_comb)
    );

    // Output register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            gray <= 'b0;
        else
            gray <= gray_comb;
    end

endmodule

```

3.11.3 Power Optimization

```

// Clock gating for power savings
module bin2gray_gated #(
    parameter int WIDTH = 8
) (
    input logic          clk,
    input logic          rst_n,
    input logic          enable,
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic gated_clk;

    // Clock gate
    clock_gate cg_inst (
        .clk(clk),
        .enable(enable),
        .gated_clk(gated_clk)
    );

    // Registered converter

```

```

    bin2gray_registered #(.WIDTH(WIDTH)) conv (
        .clk(gated_clk),
        .rst_n(rst_n),
        .binary(binary),
        .gray(gray)
    );

```

endmodule

3.12 Common Applications Summary

3.12.11. Asynchronous FIFOs: Prevent metastability in pointer comparisons

3.12.22. Clock Domain Crossing: Safe multi-bit signal transfer

3.12.33. Position Encoders: Mechanical/optical encoder interfaces

3.12.44. Memory Address Generation: Reduce EMI and power spikes

3.12.55. Test Pattern Generation: Controlled single-bit transitions

3.12.66. Error Detection: Single-bit error detection schemes

3.12.77. ADC/DAC Interfaces: Reduce glitches in conversion systems

3.13 Design Guidelines

3.13.11. Always Use for Async Boundaries: Gray code is essential for safe asynchronous transfers

3.13.22. Consider Timing: Purely combinational - no setup/hold concerns

3.13.33. Verify Single-Bit Property: Always verify adjacent values differ by one bit

3.13.44. Plan for Back-Conversion: Usually need Gray-to-Binary converter too

3.13.55. Check Width Requirements: Ensure sufficient bits for application range

The Binary-to-Gray converter is a fundamental building block in digital design, providing robust solutions for asynchronous interfaces and glitch-free operations. Its simplicity and reliability make it indispensable for safe digital system design.

3.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

4 Gray-to-Binary Converter (gray2bin.sv)

4.1 Purpose

Converts Gray code (reflected binary code) to standard binary representation using XOR reduction. Essential component for asynchronous FIFO pointer comparison and other CDC applications.

4.2 Ports

4.2.1 Input Ports

- **gray[WIDTH-1:0]** - Gray code input

4.2.2 Output Ports

- **binary[WIDTH-1:0]** - Binary code output

4.2.3 Parameters

- **WIDTH** - Data width in bits (default: 4)

4.3 Gray Code Theory

4.3.1 What is Gray Code?

Gray code (also called reflected binary code) is a binary numeral system where **only one bit changes** between consecutive values:

Binary	Gray	Transitions
000	→ 000	
001	→ 001	1 bit change
010	→ 011	1 bit change
011	→ 010	1 bit change
100	→ 110	1 bit change
101	→ 111	1 bit change
110	→ 101	1 bit change
111	→ 100	1 bit change

4.3.2 Why Gray Codes Matter for CDC

- **Metastability protection:** Only one bit transition eliminates multi-bit race conditions
- **Safe sampling:** Intermediate values during transition are still valid Gray codes
- **FIFO pointers:** Essential for async FIFO full/empty detection

4.4 Conversion Algorithm

4.4.1 Mathematical Foundation

For Gray-to-binary conversion, each binary bit is the XOR of all Gray bits from that position to the MSB:

$$\text{binary}[i] = \text{gray}[\text{MSB}] \oplus \text{gray}[\text{MSB}-1] \oplus \dots \oplus \text{gray}[i]$$

4.4.2 Implementation

```
genvar i;
generate
    for (i = 0; i < WIDTH; i++) begin : gen_gray_to_bin
        assign binary[i] = ^(gray >> i);
    end
endgenerate
```

4.4.3 Bit-by-Bit Breakdown

```
// For 4-bit example:
assign binary[3] = gray[3];           // MSB unchanged
assign binary[2] = gray[3] ^ gray[2]; // XOR from MSB
down
assign binary[1] = gray[3] ^ gray[2] ^ gray[1]; // XOR from MSB
down
assign binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0]; // XOR all
bits
```

4.5 Functional Examples

4.5.1 4-Bit Conversion Table

Gray[3:0]	Binary[3:0]	Calculation
0000	0000	$0 \oplus 0 \oplus 0 \oplus 0 = 0,$ $0 \oplus 0 \oplus 0 = 0, 0 \oplus 0 = 0,$ $0 = 0$
0001	0001	$0 \oplus 0 \oplus 0 \oplus 1 = 1,$

Gray[3:0]	Binary[3:0]	Calculation
		$0 \oplus 0 \oplus 0 = 0, 0 \oplus 0 = 0, 0 = 0$
0011	0010	$0 \oplus 0 \oplus 1 \oplus 1 = 0, 0 \oplus 0 \oplus 1 = 1, 0 \oplus 0 = 0, 0 = 0$
0010	0011	$0 \oplus 0 \oplus 1 \oplus 0 = 1, 0 \oplus 0 \oplus 1 = 1, 0 \oplus 0 = 0, 0 = 0$
0110	0100	$0 \oplus 1 \oplus 1 \oplus 0 = 0, 0 \oplus 1 \oplus 1 = 0, 0 \oplus 1 = 1, 0 = 0$
0111	0101	$0 \oplus 1 \oplus 1 \oplus 1 = 1, 0 \oplus 1 \oplus 1 = 0, 0 \oplus 1 = 1, 0 = 0$
0101	0110	$0 \oplus 1 \oplus 0 \oplus 1 = 0, 0 \oplus 1 \oplus 0 = 1, 0 \oplus 1 = 1, 0 = 0$
0100	0111	$0 \oplus 1 \oplus 0 \oplus 0 = 1, 0 \oplus 1 \oplus 0 = 1, 0 \oplus 1 = 1, 0 = 0$

4.5.2 Step-by-Step Example (Gray 0110 → Binary)

Gray input: 0110

```

binary[3] = gray[3] = 0
binary[2] = gray[3] ^ gray[2] = 0 ^ 1 = 1
binary[1] = gray[3] ^ gray[2] ^ gray[1] = 0 ^ 1 ^ 1 = 0
binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0] = 0 ^ 1 ^ 1 ^ 0 = 0

```

Result: binary = 0100 (decimal 4)

4.6 Implementation Analysis

4.6.1 XOR Reduction Technique

assign binary[i] = ^(gray >> i);

How it works: - gray >> i: Right-shift gray by i positions - ^(...): XOR-reduce all bits in the shifted value - **Effect:** XORs all bits from position i to MSB

4.6.2 Generate Loop Benefits

- **Parameterizable:** Works for any WIDTH
- **Synthesis friendly:** Tools recognize and optimize pattern
- **Parallel computation:** All bits calculated simultaneously
- **Resource efficient:** Maps to XOR tree structures

4.7 Use Cases in Digital Design

4.7.1 Asynchronous FIFO Pointers

```
// Convert synchronized Gray pointers back to binary for comparison
gray2bin #(.WIDTH(5)) wr_ptr_conv (
    .gray(sync_wr_ptr_gray),
    .binary(sync_wr_ptr_bin)
);

gray2bin #(.WIDTH(5)) rd_ptr_conv (
    .gray(sync_rd_ptr_gray),
    .binary(sync_rd_ptr_bin)
);

// Now can perform binary arithmetic for occupancy calculation
assign occupancy = sync_wr_ptr_bin - sync_rd_ptr_bin;
```

4.7.2 Clock Domain Crossing Counters

```
// Convert Gray counter back to binary for address generation
gray2bin #(.WIDTH(AW)) addr_converter (
    .gray(gray_counter),
    .binary(memory_address)
);
```

4.7.3 Position Encoding

```
// Convert Gray-encoded position to binary for processing
gray2bin #(.WIDTH(8)) position_decoder (
    .gray(gray_position),
    .binary(bin_position)
);
```

4.8 Timing Characteristics

4.8.1 Propagation Delay

- **XOR tree depth:** $\log_2(\text{WIDTH})$ levels
- **Typical delay:** 1-2 LUT delays for most widths

- **Critical path:** From MSB input to LSB output

4.8.2 Delay Analysis by Width

WIDTH	XOR Levels	Typical Delay
4	2	1 LUT delay
8	3	2 LUT delays
16	4	2 LUT delays
32	5	3 LUT delays

4.8.3 Synthesis Optimization

Modern synthesis tools: - **Recognize pattern:** Optimize XOR reduction automatically - **Balance trees:** Create balanced XOR structures - **Resource sharing:** Reuse XOR gates where possible

4.9 Design Considerations

4.9.1 Width Scaling

- **Linear resource growth:** XOR gates increase with WIDTH
- **Logarithmic delay growth:** Delay scales with $\log(\text{WIDTH})$
- **Practical limits:** Works well up to 64+ bits

4.9.2 Input Validation

```
// Gray codes have no invalid states - all inputs are valid
// However, may want to validate source of Gray code
always_comb begin
    if (gray_code_valid) begin
        binary_out = converted_binary;
    end else begin
        binary_out = '0; // Default when invalid
    end
end
```

4.9.3 Pipeline Considerations

For very high-speed applications:

```
// Optional pipeline stage for timing closure
always_ff @(posedge clk) begin
    if (!rst_n) begin
        binary_reg <= '0;
    end else begin
        binary_reg <= binary_comb;
    end
end
```

```
    end
end
```

4.10 Verification Strategies

4.10.1 Exhaustive Testing

```
// For reasonable widths, test all possible inputs
for (int i = 0; i < (1 << WIDTH); i++) begin
    gray_input = int_to_gray(i);          // Convert integer to Gray
    expected_binary = i;                  // Expected result
    #1;
    assert(binary_output == expected_binary);
end
```

4.10.2 Property-Based Verification

```
// Verify round-trip conversion
property round_trip;
    @(posedge clk)
        binary_output == original_binary;
endproperty

// Apply after binary→Gray→binary conversion
assert property (round_trip);
```

4.10.3 Random Testing

```
repeat (10000) begin
    random_gray = $random();
    expected_result = reference_gray2bin(random_gray);
    #1;
    assert(binary_output == expected_result);
end
```

4.11 Common Mistakes and Pitfalls

4.11.1 Bit Order Confusion

```
// WRONG: Bit order matters in Gray codes
assign binary[i] = ^(gray << i); // Left shift instead of right

// CORRECT:
assign binary[i] = ^(gray >> i); // Right shift
```

4.11.2 Width Mismatches

```
// WRONG: Input/output width mismatch
gray2bin #(.WIDTH(4)) converter (
    .gray(gray_5bit), // 5-bit input
```

```

        .binary(binary_4bit) // 4-bit output
    );

    // CORRECT: Match widths
    gray2bin #(.WIDTH(5)) converter (
        .gray(gray_5bit),
        .binary(binary_5bit)
    );

```

4.11.3 Timing Assumptions

```

    // WRONG: Assuming zero delay
    gray_in <= new_value;
    binary_out <= converted_value; // May not be updated yet

    // CORRECT: Account for combinational delay
    gray_in <= new_value;
    #1; // Wait for conversion
    binary_out <= converted_value;

```

4.12 Related Modules

- **bin2gray**: Performs inverse conversion (binary to Gray)
- **counter_bingray**: Combined binary/Gray counter
- **fifo_async**: Uses Gray codes for CDC
- **johnson2bin**: Johnson counter to binary converter (different algorithm)

4.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

5 Johnson-to-Binary Converter (johnson2bin.sv)

5.1 Purpose

Converts Johnson counter codes to binary representation for use in asynchronous FIFOs with non-power-of-2 depths. Unlike standard Gray-to-binary conversion, this module handles the unique properties of Johnson counter sequences.

5.2 Ports

5.2.1 Input Ports

- **clk** - Clock signal (required for leading/trailing one detection)
- **rst_n** - Active-low reset
- **gray[JCW-1:0]** - Johnson counter input (misleadingly named “gray”)

5.2.2 Output Ports

- **binary[WIDTH-1:0]** - Binary output representing position in sequence

5.2.3 Parameters

- **JCW** - Johnson Counter Width (equals FIFO DEPTH)
- **WIDTH** - Binary output width (typically $\lceil \log_2(\text{DEPTH}) \rceil + 1$)
- **INSTANCE_NAME** - String identifier for debug

5.3 Johnson Counter Sequence Review

5.3.1 Johnson Counter Characteristics

Johnson counters are **shift registers with inverted feedback**:

For JCW=6 (DEPTH=6):

```
State 0: 000000 ← Initial state
State 1: 100000 ← Shift left, insert 1
State 2: 110000
State 3: 111000
State 4: 111100
State 5: 111110
State 6: 111111 ← All 1s (special case)
State 7: 011111 ← Shift left, insert ~MSB (0)
State 8: 001111
State 9: 000111
State 10: 000011
State 11: 000001
State 0: 000000 ← Cycle complete (12 states total)
```

5.3.2 Key Properties

- **Single bit transitions**: Only one bit changes per state
- **Two phases**:
 - **First half** (0 to DEPTH-1): Filling with 1s from left
 - **Second half** (DEPTH to 2×DEPTH-1): Emptying 1s from left
- **Wrap indicator**: MSB indicates which half of cycle

5.4 Conversion Algorithm

5.4.1 Strategy Overview

The conversion uses **position detection** of the transition between 1s and 0s:

```
if (w_all_zeroes || w_all_ones) begin
    w_binary = {WIDTH{1'b0}};
end else if (gray[JCW-1]) begin
    // Second half: use leading one position directly
    w_binary = {{(WIDTH-N){1'b0}}, w_trailing_one};
end else begin
    // First half: use trailing one + 1
    w_binary = {{(WIDTH-N){1'b0}}, (w_leading_one + 1'b1)};
end
```

5.4.2 Position Detection Module

```
leading_one_trailing_one #(
    .WIDTH(JCW),
    .INSTANCE_NAME(INSTANCE_NAME)
) u_leading_one_trailing_one (
    .data(gray),
    .leadingone(w_leading_one),
    .trailingone(w_trailing_one),
    .all_zeroes(w_all_zeroes),
    .all_ones(w_all_ones),
    .valid(w_valid)
);
```

5.5 Detailed Conversion Examples

5.5.1 First Half Conversion (MSB = 0)

Johnson: 001111 (w_leading_one = 5, w_trailing_one = 2)

Logic: First half, so position = w_leading_one + 1 = 5 + 1 = 6

Binary: 000110 (with MSB=0 indicating first half)

5.5.2 Second Half Conversion (MSB = 1)

Johnson: 111000 (w_leading_one = 0, w_trailing_one = 2)

Logic: Second half, so position = w_trailing_one = 2

Binary: 100010 (with MSB=1 indicating second half)

5.5.3 Special Cases

Johnson: 000000 → Binary: 000000 (all zeros case)

Johnson: 111111 → Binary: 000000 (all ones case - same as zeros)

5.6 Implementation Deep Dive

5.6.1 Three-Part Binary Construction

```
assign binary[WIDTH-1] = gray[JCW-1];           // MSB = wrap
indicator
assign binary[WIDTH-2:0] = w_binary[WIDTH-2:0]; // Lower bits
= position
```

5.6.2 Width Calculations

```
localparam int N = $clog2(JCW);                 // Address
bits needed
localparam int PAD_WIDTH = (WIDTH > N+1) ? WIDTH-N-1 : 0; // Padding
if needed
```

5.6.3 Why This Algorithm Works

5.6.3.1 First Half Logic (MSB = 0)

- **Pattern:** 000...0111...1 (0s followed by 1s)
- **Leading one:** Position of leftmost 1
- **Conversion:** Position = leading_one + 1
- **Reasoning:** We've filled (leading_one + 1) positions with 1s

5.6.3.2 Second Half Logic (MSB = 1)

- **Pattern:** 111...1000...0 (1s followed by 0s)
- **Trailing one:** Position of rightmost 1
- **Conversion:** Position = trailing_one
- **Reasoning:** We're emptying from the left, trailing_one shows how far

5.7 Use in Asynchronous FIFO

5.7.1 Context in FIFO Operation

```
// fifo_async.sv (USE_JOHNSON=1) usage:
johnson2bin #(
    .JCW(JCW),                // = DEPTH
    .WIDTH(AW + 1),          // Address width + wrap bit
    .INSTANCE_NAME("rd_ptr_gray2bin_inst")
) rd_ptr_gray2bin_inst(
    .binary(w_wdom_rd_ptr_bin), // Binary for arithmetic
    .gray(r_wdom_rd_ptr_gray),  // Johnson counter from CDC
    .clk(wr_clk),
```

```
    .rst_n(wr_rst_n)
);
```

5.7.2 Why Clock is Required

Unlike pure combinational Gray-to-binary conversion, Johnson-to-binary needs the `leading_one_trailing_one` helper, which may use sequential logic for complex position detection.

5.8 Comparison with Standard Gray2Bin

Aspect	Standard Gray2Bin	Johnson2Bin (johnson2bin)
Input type	Traditional Gray code	Johnson counter sequence
Algorithm	XOR reduction	Position detection
Complexity	Simple combinational	Complex position logic
Clock requirement	None	Required for helper modules
Width scaling	Logarithmic	Linear with JCW
Use case	Power-of-2 sequences	Any even sequences

5.9 Performance Characteristics

5.9.1 Timing Analysis

- **Critical path:** Through position detection logic
- **Delay components:**
 - Leading/trailing one detection
 - Binary arithmetic (addition)
 - Output multiplexing
- **Synthesis complexity:** Higher than standard Gray conversion

5.9.2 Resource Utilization

Resources scale with JCW (Johnson Counter Width):

- Small FIFOs (JCW ≤ 16): Reasonable overhead
- Medium FIFOs (JCW = 32-64): Significant resources
- Large FIFOs (JCW > 64): May become limiting factor

5.10 Design Considerations

5.10.1 When to Use Johnson vs. Gray

```
// Use Johnson counter approach when:  
parameter int DEPTH = 10; // Non-power-of-2 required  
parameter int DEPTH = 6; // Specific size needed  
parameter int DEPTH = 14; // Standard Gray won't work  
  
// Use standard Gray approach when:  
parameter int DEPTH = 8; // Power-of-2 is acceptable  
parameter int DEPTH = 16; // Efficiency more important  
parameter int DEPTH = 64; // Large depth, resource conscious
```

5.10.2 Resource Planning

```
// Resource overhead estimation:  
// Standard Gray: ~log2(DEPTH) XOR gates  
// Johnson: ~DEPTH comparators + position logic  
  
// Break-even point typically around DEPTH = 16-32
```

5.10.3 Verification Challenges

```
// More complex verification due to:  
// 1. Two-phase sequence behavior  
// 2. Position detection correctness  
// 3. Special case handling (all 0s, all 1s)  
// 4. Wraparound boundary conditions
```

5.11 Error Conditions and Debug

5.11.1 Invalid Johnson Sequences

The `leading_one_trailing_one` module provides validation:

```
.valid(w_valid) // Indicates valid Johnson pattern
```

Valid Johnson patterns have exactly one transition from 1s to 0s (or vice versa).

5.11.2 Debug Visibility

```
// Monitor internal state for debugging:  
// - w_leading_one: Position of leftmost 1  
// - w_trailing_one: Position of rightmost 1  
// - w_all_zeroes: Special case flag  
// - w_all_ones: Special case flag  
// - w_valid: Sequence validity
```

5.11.3 Common Issues

1. **Invalid sequences:** Non-Johnson patterns on input
2. **Width mismatches:** JCW vs. actual Johnson counter width
3. **Phase confusion:** Misunderstanding first vs. second half logic
4. **Boundary conditions:** All-zeros and all-ones handling

5.12 Related Modules

- **counter_johnson:** Generates Johnson counter sequences
- **leading_one_trailing_one:** Position detection helper
- **fifo_async / gaxi_fifo_async (USE_JOHNSON=1):** Primary users of this conversion
- **gray2bin:** Standard Gray-to-binary for comparison
- **fifo_control:** Uses converted binary values for status generation

5.13 Advanced Topics

5.13.1 Hierarchical Johnson Counters

For very large depths, consider hierarchical approach:

```
// Break large Johnson counter into smaller segments
// Use multiple johnson2bin instances with higher-level arbitration
```

5.13.2 Alternative Position Detection

Some implementations use different position detection algorithms: - **Priority encoders:** Find first/last set bit - **Thermometer decoders:** Convert 1-hot positions - **LUT-based:** For small, fixed widths

5.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

6 Binary-Gray Counter Module

6.1 Overview

The counter_bingray module is a dual-output counter that simultaneously provides both binary and Gray code representations of the same count value. This is essential for asynchronous FIFO

implementations where Gray code is required to prevent metastability when crossing clock domains, while binary values are needed for arithmetic operations.

6.2 Module Declaration

```
module counter_bingray #(
    parameter int WIDTH = 4
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    output logic [WIDTH-1:0] counter_bin,
    output logic [WIDTH-1:0] counter_bin_next,
    output logic [WIDTH-1:0] counter_gray
);
```

6.3 Parameters

6.3.1 WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Bit width of both binary and Gray code outputs
- **Range:** Any positive integer ≥ 1
- **Impact:** Determines maximum count value ($2^{\text{WIDTH}} - 1$)

6.4 Ports

6.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous reset
enable	1	logic	Counter enable control

6.4.2 Outputs

Port	Width	Type	Description
counter_bin	WIDTH	logic	Binary counter output

Port	Width	Type	Description
			(registered)
counter_bin_next	WIDTH	logic	Next binary value (combinational)
counter_gray	WIDTH	logic	Gray code output (registered)

6.5 Architecture Details

6.5.1 Internal Logic

```
logic [WIDTH-1:0] w_counter_bin, w_counter_gray;
```

```
assign w_counter_bin = enable ? (counter_bin + 1) : counter_bin;
```

```
assign w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
```

```
assign counter_bin_next = w_counter_bin;
```

6.5.2 Gray Code Conversion

The module implements the standard binary-to-Gray conversion: - **MSB**: $\text{gray}[\text{WIDTH}-1] = \text{binary}[\text{WIDTH}-1]$ - **Other bits**: $\text{gray}[i] = \text{binary}[i] \oplus \text{binary}[i+1]$ for $i = 0$ to $\text{WIDTH}-2$

6.6 Gray Code Theory

6.6.1 Why Gray Code?

Gray code (reflected binary code) has the property that adjacent values differ by exactly one bit. This is crucial for:

1. **Metastability Prevention:** When crossing clock domains, only one bit changes at a time
2. **Glitch Elimination:** Reduces intermediate states during transitions
3. **Asynchronous Safety:** Safe for use in asynchronous circuits

6.6.2 Gray Code Sequence Example (4-bit)

Decimal	Binary	Gray	Changes
0	0000	0000	-
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2

Decimal	Binary	Gray	Changes
5	0101	0111	bit 0
6	0110	0101	bit 1
7	0111	0100	bit 0
8	1000	1100	bit 3
9	1001	1101	bit 0
10	1010	1111	bit 1
11	1011	1110	bit 0
12	1100	1010	bit 2
13	1101	1011	bit 0
14	1110	1001	bit 1
15	1111	1000	bit 0

6.7 Implementation Analysis

6.7.1 Combinational Logic

```
// Next binary calculation
w_counter_bin = enable ? (counter_bin + 1) : counter_bin;

// Gray code conversion
w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
```

6.7.2 Sequential Logic

```
always_ff @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
        counter_bin <= 'b0;
        counter_gray <= 'b0;
    end else begin
        counter_bin <= w_counter_bin;
        counter_gray <= w_counter_gray;
    end
end
```

6.8 Timing Characteristics

6.8.1 Critical Paths

1. **Binary Increment:** Standard binary addition timing
2. **Gray Conversion:** XOR tree with $\log(\text{WIDTH})$ levels

3. **Combined Path:** Increment + conversion in same cycle

6.8.2 Propagation Delays

- **Clock-to-Q:** Standard flip-flop delay
- **Combinational:** Depends on addition and XOR logic depth
- **Setup Time:** Must account for longest combinational path

6.9 Application: Asynchronous FIFO

6.9.1 FIFO Pointer Implementation

```
// Write domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) wr_counter (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(wr_enable),
    .counter_bin(wr_bin),
    .counter_bin_next(wr_bin_next),
    .counter_gray(wr_gray)
);

// Read domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) rd_counter (
    .clk(rd_clk),
    .rst_n(rd_rst_n),
    .enable(rd_enable),
    .counter_bin(rd_bin),
    .counter_bin_next(rd_bin_next),
    .counter_gray(rd_gray)
);
```

6.9.2 Cross-Domain Synchronization

```
// Synchronize Gray code pointers across domains
logic [ADDR_WIDTH:0] wr_gray_sync, rd_gray_sync;

// Write domain: synchronize read Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) rd_sync (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .data_in(rd_gray),
    .data_out(rd_gray_sync)
);

// Read domain: synchronize write Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) wr_sync (
```

```

        .clk(rd_clk),
        .rst_n(rd_rst_n),
        .data_in(wr_gray),
        .data_out(wr_gray_sync)
    );

```

6.9.3 FIFO Status Generation

```

// FIFO empty: Gray pointers equal
assign fifo_empty = (rd_gray == wr_gray_sync);

// FIFO full: Binary addresses equal, MSBs different
wire [ADDR_WIDTH-1:0] wr_addr = wr_bin[ADDR_WIDTH-1:0];
wire [ADDR_WIDTH-1:0] rd_addr_sync = gray2bin(rd_gray_sync)
[ADDR_WIDTH-1:0];
wire wr_msb = wr_bin[ADDR_WIDTH];
wire rd_msb_sync = gray2bin(rd_gray_sync)[ADDR_WIDTH];

assign fifo_full = (wr_addr == rd_addr_sync) && (wr_msb !=
rd_msb_sync);

```

6.10 Advanced Features

6.10.1 Almost Full/Empty Flags

```

// Calculate occupancy using binary values
wire [ADDR_WIDTH:0] occupancy = wr_bin - gray2bin(rd_gray_sync);

// Generate status flags
assign almost_full = (occupancy >= ALMOST_FULL_THRESH);
assign almost_empty = (occupancy <= ALMOST_EMPTY_THRESH);

```

6.10.2 Gray-to-Binary Conversion Function

```

function automatic [WIDTH-1:0] gray2bin;
    input [WIDTH-1:0] gray;
    integer i;
    begin
        gray2bin[WIDTH-1] = gray[WIDTH-1];
        for (i = WIDTH-2; i >= 0; i--) begin
            gray2bin[i] = gray2bin[i+1] ^ gray[i];
        end
    end
endfunction

```

6.11 Design Examples

6.11.1 8-bit Asynchronous FIFO Pointer

```
parameter FIFO_DEPTH = 256;
parameter ADDR_WIDTH = $clog2(FIFO_DEPTH);
parameter PTR_WIDTH = ADDR_WIDTH + 1; // Extra bit for full detection
```

```
counter_bingray #(
    .WIDTH(PTR_WIDTH)
) fifo_wr_ptr (
    .clk(wr_clk),
    .rst_n(async_rst_n),
    .enable(wr_enable && !fifo_full),
    .counter_bin(wr_ptr_bin),
    .counter_bin_next(wr_ptr_bin_next),
    .counter_gray(wr_ptr_gray)
);
```

6.11.22. Clock Domain Crossing Counter

```
// Source domain
```

```
counter_bingray #(.WIDTH(8)) src_counter (
    .clk(src_clk),
    .rst_n(src_rst_n),
    .enable(src_enable),
    .counter_bin(src_bin),
    .counter_bin_next(),
    .counter_gray(src_gray)
);
```

```
// Destination domain receives synchronized Gray value
```

```
logic [7:0] dest_gray_sync;
synchronizer #(.WIDTH(8)) sync_inst (
    .clk(dest_clk),
    .rst_n(dest_rst_n),
    .data_in(src_gray),
    .data_out(dest_gray_sync)
);
```

6.12 Verification Strategy

6.12.1 Test Scenarios

1. **Sequential Counting:** Verify both outputs increment correctly
2. **Gray Code Properties:** Ensure single-bit changes between adjacent values
3. **Reset Behavior:** Confirm both outputs reset to zero

4. **Enable Control:** Test hold behavior when disabled
5. **Rollover:** Verify proper wrap-around from maximum value

6.12.2 Coverage Points

```
covergroup counter_bingray_cg @(posedge clk);
  cp_binary: coverpoint counter_bin {
    bins all_values[] = {[0:2**WIDTH-1]};
  }

  cp_gray: coverpoint counter_gray {
    bins all_values[] = {[0:2**WIDTH-1]};
  }

  cp_enable: coverpoint enable {
    bins enabled = {1};
    bins disabled = {0};
  }

  cp_transitions: coverpoint counter_bin {
    bins transitions[] = ([0:2**WIDTH-2] => [1:2**WIDTH-1]);
    bins rollover = (2**WIDTH-1 => 0);
  }
endgroup
```

6.12.3 Assertions

```
// Verify Gray code has single bit changes
property gray_single_bit_change;
  @(posedge clk) disable iff (!rst_n)
  enable && !$isunknown(counter_gray) |->
  $countones(counter_gray ^ $past(counter_gray)) <= 1;
endproperty

assert property (gray_single_bit_change);

// Verify binary and Gray relationship
property bin_gray_relationship;
  @(posedge clk) disable iff (!rst_n)
  counter_gray == (counter_bin ^ (counter_bin >> 1));
endproperty

assert property (bin_gray_relationship);
```

6.13 Synthesis Considerations

6.13.1 Resource Utilization

- **Flip-Flops:** 2×WIDTH (separate for binary and Gray)
- **LUTs:** Increment logic + XOR tree for Gray conversion
- **Critical Path:** Through binary increment and Gray conversion

6.13.2 Optimization Techniques

```
// Optional: Pipeline Gray conversion for high speed  
logic [WIDTH-1:0] counter_gray_pipe;  
always_ff @(posedge clk) begin  
    counter_gray_pipe <= w_counter_gray;  
end
```

6.13.3 Tool-Specific Attributes

```
(* ASYNC_REG = "TRUE" *) logic [WIDTH-1:0] counter_gray; // Xilinx  
// synthesis attribute ASYNC_REG of counter_gray is "TRUE" //  
Altera/Intel
```

6.14 Performance Characteristics

6.14.1 Maximum Frequency

- **Typical:** 200-400 MHz in modern FPGAs
- **Limitation:** Binary increment + Gray conversion logic depth
- **Optimization:** Pipeline for higher frequencies

6.14.2 Power Consumption

- **Dynamic:** Proportional to switching activity
- **Static:** Minimal for CMOS technology
- **Optimization:** Clock gating when enable is inactive

6.15 Common Applications

1. **Asynchronous FIFO Pointers:** Primary use case
2. **Clock Domain Crossing Counters:** Safe multi-domain counting
3. **Position Encoders:** Mechanical/optical encoder interfaces
4. **State Machine Counters:** When glitch-free operation required
5. **Memory Address Generation:** For dual-port memory systems

6.16 Troubleshooting Guide

6.16.1 Common Issues

1. **Metastability:** Ensure proper synchronization of Gray values
2. **Timing Violations:** Pipeline Gray conversion if needed
3. **Incorrect FIFO Status:** Verify Gray-to-binary conversion
4. **Reset Skew:** Use proper reset synchronization

6.16.2 Debug Techniques

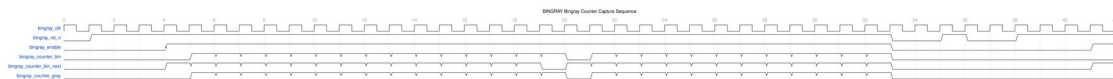
1. **Simulation:** Verify Gray code properties with assertions
2. **Logic Analyzer:** Capture actual transitions in hardware
3. **Timing Analysis:** Check setup/hold requirements
4. **Cross-Domain Checks:** Verify synchronizer timing

6.17 WaveDrom Visualization

High-quality waveforms showcasing Binary-Gray counter operation are available!

The following timing diagrams demonstrate the dual-output counter design across 4 key scenarios:

6.17.1 Scenario 1: Binary vs Gray Code Comparison

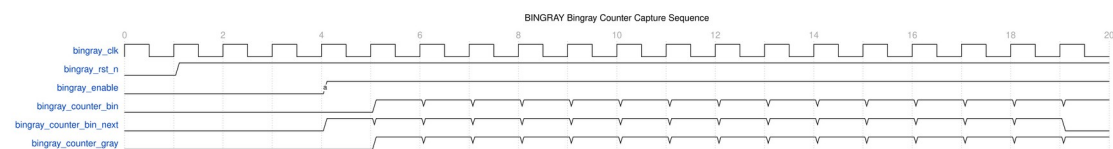


BinGray Comparison

WaveJSON: [bingray_counter_binary_vs_gray.json](#)

Side-by-side comparison of both outputs: - Binary: Normal sequential counting (0→1→2→3→...) - Gray: Single-bit transitions between values - Shows full cycle demonstrating encoding differences - Illustrates why Gray code is CDC-safe

6.17.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE

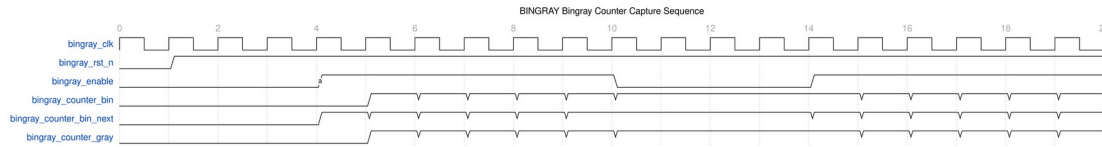


BinGray Single-Bit Transitions

WaveJSON: [bingray_counter_single_bit_transitions.json](#)

Gray code CDC safety property: - Each Gray transition changes EXACTLY one bit - Hamming distance = 1 between adjacent values - **Critical for fifo_async CDC mechanism** - Prevents metastability in clock domain crossing

6.17.3 Scenario 3: Lookahead Signal (counter_bin_next)

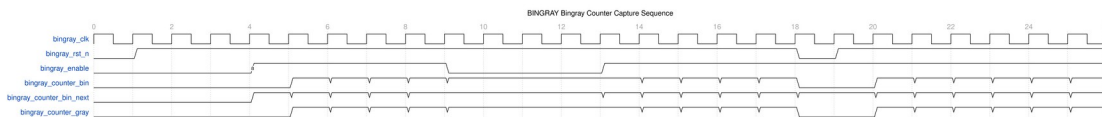


BinGray Lookahead

WaveJSON: [bingray_counter_lookahead.json](#)

Combinational lookahead feature: - Predicts next binary value one cycle ahead - Useful for FIFO full/empty prediction - Shows enable gating behavior - Enables early decision-making

6.17.4 Scenario 4: Enable and Reset Control



BinGray Enable and Reset

WaveJSON: [bingray_counter_enable_reset.json](#)

Control signal behavior: - Both outputs hold when enable=0 - Asynchronous reset to zero - Clean state transitions - Reset during counting demonstration

To regenerate these waveforms:

```
pytest val/common/test_counter_bingray_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/WAVES/counter_bingray
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done
```

What Makes Binary-Gray Counters Special:

The waveforms highlight the unique dual-output design: - **Dual Encoding:** Binary for arithmetic, Gray for CDC safety - **Single-Bit Transitions:** Gray code changes one bit at a time - **Lookahead:** counter_bin_next provides one-cycle prediction - **CDC-Safe:** Safe for asynchronous clock domain crossing

Relationship to fifo_async:

Binary-Gray counters are the foundation of the standard `fifo_async` module: - **fifo_async** uses this counter for read/write pointers - Gray outputs cross clock domains safely - Binary outputs used for arithmetic (occupancy, address generation) - Logarithmic width ($\lceil \log_2(\text{DEPTH}) \rceil + 1$) for resource efficiency

Comparison Table:

Feature	BinGray Counter	Johnson Counter
Output Width	$\lceil \log_2(\text{DEPTH}) \rceil + 1$ bits	DEPTH bits
CDC Method	Standard Gray code	Johnson code
Depth Support	Power-of-2 only	Any even number
Resource Efficiency	Logarithmic (better for large depths)	Linear
Conversion	XOR tree (simple)	Position detection (complex)
Used In	<code>fifo_async USE_JOHNSON=0</code>	<code>fifo_async USE_JOHNSON=1</code>

Comparison with Other Modules:

- `test_counter_johnson_wavedrom.py` - Johnson counter (even depths, linear width)
- `test_fifo_async_wavedrom.py` - BinGray counter in action (async FIFO, power-of-2)

6.18 Test and Verification

Comprehensive Test Suite: - `val/common/test_counter_bingray.py` - Full functional verification - `val/common/test_counter_bingray_wavedrom.py` - WaveDrom timing diagrams



Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_counter_bingray.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_counter_bingray_wavedrom.py -v
```

6.19 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

7 Clock Pulse Generator - Comprehensive Documentation

7.1 Overview

The `clock_pulse` module generates periodic pulse signals with configurable width (period). It creates a single-cycle pulse every `WIDTH` clock cycles, making it ideal for timing generation, heartbeat signals, sampling triggers, and periodic event generation in digital systems.

7.2 Module Declaration

```
module clock_pulse #(
    parameter int WIDTH = 10 // Width of the generated pulse in clock
    cycles
) (
    input logic clk, // Input clock signal
    input logic rst_n, // Input reset signal
    output logic pulse // Output pulse signal
);
```

7.3 Parameters

7.3.1 WIDTH

- **Type:** int
- **Default:** 10
- **Description:** Period of the pulse generation in clock cycles
- **Range:** 2 to $2^{32}-1$ (practical range)
- **Impact:** Determines pulse frequency ($f_{clk} / WIDTH$)
- **Pulse Width:** Always exactly 1 clock cycle
- **Duty Cycle:** $1/WIDTH$ (e.g., 10% for $WIDTH=10$)

7.4 Ports

7.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous

Port	Width	Type	Description
			reset

7.4.2 Outputs

Port	Width	Type	Description
pulse	1	logic	Periodic pulse output

7.5 Architecture and Implementation

7.5.1 Internal Counter

```

logic [WIDTH-1:0] r_counter;
logic [WIDTH-1:0] w_width_minus_one;

// Create a properly sized constant
assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

```

7.5.2 Core Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse <= 'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end
end

```

7.5.3 Operation Principles

1. **Counter:** Free-running counter from 0 to WIDTH-1
2. **Pulse Generation:** Pulse asserted when counter reaches WIDTH-1
3. **Auto-Reset:** Counter wraps to 0 after reaching maximum
4. **Synchronous:** All operations synchronized to input clock
5. **Single Cycle:** Pulse duration is exactly one clock cycle

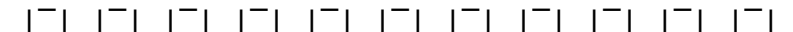
7.5.4 Timing Characteristics

- **Period:** WIDTH clock cycles

- **Frequency:** $f_{\text{clk}} / \text{WIDTH}$
- **Pulse Width:** 1 clock cycle
- **Duty Cycle:** $1/\text{WIDTH}$
- **Phase:** Pulse occurs on last count ($\text{WIDTH}-1$)

7.6 Timing Diagrams

7.6.1 Basic Operation (WIDTH=4)

Clock: 
 Counter: < 0 > < 1 > < 2 > < 3 > < 0 > < 1 > < 2 > < 3 > < 0 >
 Pulse: 

7.6.2 Reset Behavior

Clock: 
 Reset_n: 
 Counter: < 0 > < 1 > < 2 > < 3 > < 0 > < 0 > < 1 > < 2 > < 3 >
 Pulse: 

7.6.3 Different WIDTH Values

WIDTH	Period	Duty Cycle	Use Case
2	2 cycles	50%	Clock divider
4	4 cycles	25%	Sampling
10	10 cycles	10%	Timing reference
100	100 cycles	1%	Heartbeat
1000	1000 cycles	0.1%	Slow events

7.7 Design Examples and Applications

7.7.1 1. Heartbeat and Status Indicators

```

module system_heartbeat #(
    parameter int CLOCK_FREQ_HZ = 100_000_000, // 100MHz
    parameter int HEARTBEAT_FREQ_HZ = 1        // 1Hz heartbeat
) (
    input logic clk,
    input logic rst_n,
    input logic system_active,
    output logic heartbeat_led,
    output logic status_ok
);

```

```

    localparam int HEARTBEAT_WIDTH = CLOCK_FREQ_HZ /
HEARTBEAT_FREQ_HZ;

    logic heartbeat_pulse;

    // Generate 1Hz heartbeat pulse
    clock_pulse #(
        .WIDTH(HEARTBEAT_WIDTH)
    ) heartbeat_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(heartbeat_pulse)
    );

    // LED control with heartbeat
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            heartbeat_led <= 1'b0;
        end else if (heartbeat_pulse) begin
            heartbeat_led <= ~heartbeat_led; // Toggle LED
        end
    end

    // System status based on heartbeat activity
    logic [7:0] heartbeat_monitor;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            heartbeat_monitor <= 8'h00;
        end else begin
            heartbeat_monitor <= {heartbeat_monitor[6:0],
heartbeat_pulse};
        end
    end

    assign status_ok = system_active && |heartbeat_monitor;

endmodule

```

7.7.2 2. Sampling and Data Acquisition

```

module data_sampler #(
    parameter int SAMPLE_RATE_KHZ = 48,
    parameter int CLOCK_FREQ_MHZ = 100,
    parameter int DATA_WIDTH = 16
) (
    input logic          clk,

```

```

    input  logic          rst_n,
    input  logic          enable,
    input  logic [DATA_WIDTH-1:0] analog_data,

    output logic [DATA_WIDTH-1:0] sampled_data,
    output logic          sample_valid,
    output logic          buffer_full
);

    localparam int SAMPLE_WIDTH = (CLOCK_FREQ_MHZ * 1000) /
SAMPLE_RATE_KHZ;

    logic sample_trigger;

    // Generate sampling trigger
    clock_pulse #(
        .WIDTH(SAMPLE_WIDTH)
    ) sample_clock (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(sample_trigger)
    );

    // Sample data on trigger
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            sampled_data <= '0;
            sample_valid <= 1'b0;
        end else if (enable && sample_trigger) begin
            sampled_data <= analog_data;
            sample_valid <= 1'b1;
        end else begin
            sample_valid <= 1'b0;
        end
    end

    // Buffer management (simplified)
    logic [7:0] buffer_count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            buffer_count <= 8'h00;
        end else if (sample_valid) begin
            buffer_count <= buffer_count + 1;
        end
    end

    assign buffer_full = &buffer_count; // Full at 255 samples

```

```
endmodule
```

7.7.3 3. Communication Protocol Timing

```
module uart_baud_generator #(
    parameter int CLOCK_FREQ_HZ = 100_000_000,
    parameter int BAUD_RATE = 115200
) (
    input logic clk,
    input logic rst_n,
    input logic enable,

    output logic baud_tick,          // 1x baud rate
    output logic baud_tick_16x      // 16x baud rate for oversampling
);

    localparam int BAUD_WIDTH = CLOCK_FREQ_HZ / BAUD_RATE;
    localparam int BAUD_16X_WIDTH = CLOCK_FREQ_HZ / (BAUD_RATE * 16);

    logic baud_pulse, baud_16x_pulse;

    // 1x baud rate generator
    clock_pulse #(
        .WIDTH(BAUD_WIDTH)
    ) baud_1x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_pulse)
    );

    // 16x baud rate generator
    clock_pulse #(
        .WIDTH(BAUD_16X_WIDTH)
    ) baud_16x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_16x_pulse)
    );

    // Gate with enable
    assign baud_tick = enable ? baud_pulse : 1'b0;
    assign baud_tick_16x = enable ? baud_16x_pulse : 1'b0;

endmodule
```

7.7.4 4. Memory Refresh Controller

```
module dram_refresh_controller #(
    parameter int REFRESH_PERIOD_US = 64,           // 64µs refresh period
    parameter int CLOCK_FREQ_MHZ = 100,           // 100MHz system clock
    parameter int REFRESH_ROWS = 8192             // Number of rows to
refresh
) (
    input logic      clk,
    input logic      rst_n,
    input logic      refresh_enable,

    output logic      refresh_request,
    output logic [12:0] refresh_row_addr,
    output logic      refresh_active
);

    localparam int REFRESH_WIDTH = (REFRESH_PERIOD_US *
CLOCK_FREQ_MHZ) / REFRESH_ROWS;

    logic refresh_pulse;
    logic [12:0] row_counter;

    // Generate refresh timing
    clock_pulse #(
        .WIDTH(REFRESH_WIDTH)
    ) refresh_timer (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(refresh_pulse)
    );

    // Row address counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            row_counter <= 13'h0000;
        end else if (refresh_pulse && refresh_enable) begin
            if (row_counter >= REFRESH_ROWS - 1)
                row_counter <= 13'h0000;
            else
                row_counter <= row_counter + 1;
        end
    end

    // Output assignments
    assign refresh_request = refresh_pulse && refresh_enable;
    assign refresh_row_addr = row_counter;
    assign refresh_active = refresh_enable;
```

```
endmodule
```

7.7.5 5. Multi-Rate Pulse Generation

```
module multi_rate_pulse_generator (  
    input logic clk,  
    input logic rst_n,  
    input logic [2:0] rate_select,    // Select pulse rate  
  
    output logic pulse_fast,    // Fast pulse output  
    output logic pulse_medium,  // Medium pulse output  
    output logic pulse_slow,    // Slow pulse output  
    output logic pulse_config    // Configurable pulse output  
);  
  
    logic [31:0] config_width;  
  
    // Rate selection for configurable output  
    always_comb begin  
        case (rate_select)  
            3'b000: config_width = 32'd10;    // Very fast  
            3'b001: config_width = 32'd100;    // Fast  
            3'b010: config_width = 32'd1000;    // Medium  
            3'b011: config_width = 32'd10000;    // Slow  
            3'b100: config_width = 32'd100000;    // Very slow  
            default: config_width = 32'd1000;    // Default medium  
        endcase  
    end  
  
    // Fixed rate pulse generators  
    clock_pulse #(.WIDTH(50)) fast_gen (  
        .clk(clk), .rst_n(rst_n), .pulse(pulse_fast)  
    );  
  
    clock_pulse #(.WIDTH(500)) medium_gen (  
        .clk(clk), .rst_n(rst_n), .pulse(pulse_medium)  
    );  
  
    clock_pulse #(.WIDTH(5000)) slow_gen (  
        .clk(clk), .rst_n(rst_n), .pulse(pulse_slow)  
    );  
  
    // Configurable rate generator (requires parameterizable module)  
    configurable_pulse_gen config_gen (  
        .clk(clk),  
        .rst_n(rst_n),  
        .width_config(config_width),
```

```

        .pulse(pulse_config)
    );

endmodule

// Configurable pulse generator module
module configurable_pulse_gen #(
    parameter int MAX_WIDTH = 1000000
) (
    input logic [31:0] width_config,
    input logic        clk,
    input logic        rst_n,
    output logic        pulse
);

    logic [31:0] r_counter;
    logic [31:0] w_width_minus_one;

    assign w_width_minus_one = (width_config > 0) ? (width_config - 1)
    : 0;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 32'h00000000;
            pulse <= 1'b0;
        end else begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else
                r_counter <= 32'h00000000;

            pulse <= (r_counter == w_width_minus_one) && (width_config
> 0);
        end
    end

endmodule

```

7.7.6 6. Test Pattern Generation

```

module test_pattern_generator (
    input logic        clk,
    input logic        rst_n,
    input logic        test_enable,
    input logic [3:0] pattern_select,

    output logic [7:0] test_data,
    output logic        test_valid,

```

```

    output logic          pattern_complete
);

logic update_pulse;
logic [7:0] pattern_counter;

// Generate test data update rate
clock_pulse #(.WIDTH(16)) pattern_clock (
    .clk(clk),
    .rst_n(rst_n),
    .pulse(update_pulse)
);

// Pattern counter
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pattern_counter <= 8'h00;
    end else if (test_enable && update_pulse) begin
        pattern_counter <= pattern_counter + 1;
    end
end

// Test pattern generation
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        test_data <= 8'h00;
        test_valid <= 1'b0;
    end else if (test_enable && update_pulse) begin
        case (pattern_select)
            4'h0: test_data <= pattern_counter;
// Incrementing
            4'h1: test_data <= ~pattern_counter;
// Decrementing
            4'h2: test_data <= {pattern_counter[0], 7'b00000000};
// Walking 1
            4'h3: test_data <= ~{pattern_counter[0], 7'b00000000};
// Walking 0
            4'h4: test_data <= 8'hAA;
// Alternating
            4'h5: test_data <= 8'h55;
// Alternating inv
            4'h6: test_data <= 8'hFF;
// All ones
            4'h7: test_data <= 8'h00;
// All zeros
            default: test_data <= pattern_counter;
// Default
        endcase
    end
end

```

```

        test_valid <= 1'b1;
    end else begin
        test_valid <= 1'b0;
    end
end

    assign pattern_complete = test_enable && (&pattern_counter) &&
update_pulse;

endmodule

```

7.8 Advanced Features and Variations

7.8.1 1. Enable-Controlled Pulse Generator

```

module pulse_generator_with_enable #(
    parameter int WIDTH = 10
) (
    input logic clk,
    input logic rst_n,
    input logic enable,           // Pulse generation enable
    input logic pause,           // Pause counting (hold current state)
    output logic pulse,
    output logic [WIDTH-1:0] count_value // Current counter value
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;
    assign count_value = r_counter;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else if (enable && !pause) begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1'b1;
            else
                r_counter <= 'b0;

            pulse <= (r_counter == w_width_minus_one);
        end else begin
            pulse <= 1'b0; // No pulse when disabled or paused
        end
    end
end

```

endmodule

7.8.2 2. Variable Width Pulse Generator

```
module variable_pulse_generator #(
    parameter int MAX_WIDTH = 65536
) (
    input logic clk,
    input logic rst_n,
    input logic [$clog2(MAX_WIDTH):0] pulse_width, // Configurable
width
    input logic load_width, // Load new
width
    output logic pulse,
    output logic width_loaded
);

    logic [$clog2(MAX_WIDTH):0] r_counter;
    logic [$clog2(MAX_WIDTH):0] r_width;
    logic [$clog2(MAX_WIDTH):0] w_width_minus_one;

    assign w_width_minus_one = (r_width > 0) ? (r_width - 1) : 0;

    // Width register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_width <= MAX_WIDTH[$clog2(MAX_WIDTH):0]; // Default
width
            width_loaded <= 1'b0;
        end else if (load_width) begin
            r_width <= (pulse_width > 0) ? pulse_width : 1; //
Minimum width = 1
            width_loaded <= 1'b1;
        end else begin
            width_loaded <= 1'b0;
        end
    end

    // Counter and pulse generation
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else

```

```

        r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one) && (r_width >
0);
    end
end

endmodule

```

7.8.3 3. Multi-Phase Pulse Generator

```

module multi_phase_pulse_generator #(
    parameter int WIDTH = 12,
    parameter int PHASES = 4
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    output logic [PHASES-1:0] phase_pulses
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;
    logic [WIDTH-1:0] phase_thresholds [PHASES];

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

    // Calculate phase thresholds
    genvar i;
    generate
        for (i = 0; i < PHASES; i++) begin : calc_phases
            assign phase_thresholds[i] = (WIDTH * i) / PHASES;
        end
    endgenerate

    // Counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
        end else if (enable) begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else
                r_counter <= 'b0;
        end
    end

    // Phase pulse generation

```

```

generate
    for (i = 0; i < PHASES; i++) begin : gen_phases
        always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n) begin
                phase_pulses[i] <= 1'b0;
            end else begin
                phase_pulses[i] <= enable && (r_counter ==
phase_thresholds[i]);
            end
        end
    end
endgenerate

endmodule

```

7.8.4 4. Burst Pulse Generator

```

module burst_pulse_generator #(
    parameter int BURST_LENGTH = 8,
    parameter int BURST_PERIOD = 100,
    parameter int INTER_BURST_DELAY = 1000
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    input logic trigger,           // Start burst sequence

    output logic burst_pulse,      // Individual pulses in burst
    output logic burst_active,     // Burst sequence active
    output logic burst_complete   // Burst sequence completed
);

typedef enum logic [2:0] {
    IDLE,
    BURST_ACTIVE,
    INTER_PULSE_DELAY,
    INTER_BURST_DELAY_ST
} burst_state_t;

burst_state_t state;

logic [$clog2(BURST_LENGTH):0] pulse_count;
logic [$clog2(BURST_PERIOD):0] period_counter;
logic [$clog2(INTER_BURST_DELAY):0] delay_counter;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;

```

```

pulse_count <= 0;
period_counter <= 0;
delay_counter <= 0;
burst_pulse <= 1'b0;
burst_complete <= 1'b0;
end else begin
burst_complete <= 1'b0; // Default

case (state)
IDLE: begin
burst_pulse <= 1'b0;
if (enable && trigger) begin
state <= BURST_ACTIVE;
pulse_count <= 0;
period_counter <= 0;
burst_pulse <= 1'b1; // First pulse
end
end

BURST_ACTIVE: begin
burst_pulse <= 1'b0;
pulse_count <= pulse_count + 1;

if (pulse_count >= BURST_LENGTH - 1) begin
state <= INTER_BURST_DELAY_ST;
delay_counter <= 0;
burst_complete <= 1'b1;
end else begin
state <= INTER_PULSE_DELAY;
period_counter <= 0;
end
end

INTER_PULSE_DELAY: begin
period_counter <= period_counter + 1;
if (period_counter >= BURST_PERIOD - 1) begin
state <= BURST_ACTIVE;
burst_pulse <= 1'b1;
end
end

INTER_BURST_DELAY_ST: begin
delay_counter <= delay_counter + 1;
if (delay_counter >= INTER_BURST_DELAY - 1) begin
state <= IDLE;
end
end
endcase

```

```

        end
    end

    assign burst_active = (state != IDLE);

endmodule

```

7.9 Verification and Testing

7.9.1 Comprehensive Test Bench

```

module tb_clock_pulse;

    parameter WIDTH = 10;

    logic clk, rst_n, pulse;

    // Clock generation
    initial clk = 0;
    always #5ns clk = ~clk; // 100MHz

    // DUT instantiation
    clock_pulse #(.WIDTH(WIDTH)) dut (.*);

    // Test sequence
    initial begin
        rst_n = 0;
        #100ns rst_n = 1;

        // Test basic pulse generation
        test_basic_pulse_generation();

        // Test reset behavior
        test_reset_behavior();

        // Test timing accuracy
        test_timing_accuracy();

        $display("All tests completed!");
        $finish;
    end

    // Test basic pulse generation
    task test_basic_pulse_generation();
        int pulse_count = 0;
        int cycle_count = 0;
        begin
            $display("Testing basic pulse generation (WIDTH=%0d)...",

```

```

WIDTH);

    // Monitor for several pulse periods
    repeat (WIDTH * 3) begin
        @(posedge clk);
        cycle_count++;
        if (pulse) pulse_count++;
    end

    // Verify pulse count
    if (pulse_count == 3) begin
        $display("PASS: Correct number of pulses generated");
    end else begin
        $error("FAIL: Expected 3 pulses, got %0d",
pulse_count);
    end
end
endtask

// Test timing accuracy
task test_timing_accuracy();
    time pulse_times[10];
    time pulse_periods[9];
    int i;
    real avg_period, expected_period;
    begin
        $display("Testing timing accuracy...");

        expected_period = WIDTH * 10.0; // 10ns clock period

        // Capture pulse timing
        for (i = 0; i < 10; i++) begin
            @(posedge pulse);
            pulse_times[i] = $time;
        end

        // Calculate periods
        for (i = 0; i < 9; i++) begin
            pulse_periods[i] = pulse_times[i+1] - pulse_times[i];
        end

        // Calculate average period
        avg_period = 0;
        for (i = 0; i < 9; i++) begin
            avg_period += pulse_periods[i];
        end
        avg_period = avg_period / 9.0;
    end
endtask

```

```

        // Check accuracy (within 1% tolerance)
        if (abs(avg_period - expected_period) < expected_period *
0.01) begin
            $display("PASS: Average period = %.1f ns (expected
%.1f ns)",
                    avg_period, expected_period);
        end else begin
            $error("FAIL: Average period = %.1f ns (expected %.1f
ns)",
                    avg_period, expected_period);
        end
    end
endtask

function real abs(real value);
    abs = (value >= 0) ? value : -value;
endfunction

endmodule

```

7.9.2 Coverage Model

```

covergroup clock_pulse_cg @(posedge clk);

    cp_counter_values: coverpoint dut.r_counter {
        bins zero = {0};
        bins low[] = {[1:WIDTH/4]};
        bins mid[] = {[WIDTH/4+1:3*WIDTH/4]};
        bins high[] = {[3*WIDTH/4+1:WIDTH-2]};
        bins max = {WIDTH-1};
    }

    cp_pulse: coverpoint pulse {
        bins asserted = {1};
        bins deasserted = {0};
    }

    cp_reset: coverpoint rst_n {
        bins reset = {0};
        bins normal = {1};
    }

    // Transition coverage
    cp_pulse_edges: coverpoint pulse {
        bins rising = (0 => 1);
        bins falling = (1 => 0);
        bins stable_low = (0 => 0);
        bins stable_high = (1 => 1);
    }

```

```

// Cross coverage
cross_counter_pulse: cross cp_counter_values, cp_pulse;

endcovergroup

```

7.9.3 Formal Properties

```

module clock_pulse_properties;

    // Bind to DUT
    bind clock_pulse clock_pulse_properties props_inst (.*);

    // Property: Pulse occurs at correct count
    property pulse_at_max_count;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |-> pulse;
    endproperty

    // Property: Pulse only occurs at max count
    property pulse_only_at_max;
        @(posedge clk) disable iff (!rst_n)
        pulse |-> (dut.r_counter == WIDTH-1);
    endproperty

    // Property: Counter wraps correctly
    property counter_wrap;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |=> (dut.r_counter == 0);
    endproperty

    // Property: Counter increments
    property counter_increment;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter < WIDTH-1) |=>
        (dut.r_counter == $past(dut.r_counter) + 1);
    endproperty

    // Property: Reset behavior
    property reset_behavior;
        @(posedge clk)
        !rst_n |=> (dut.r_counter == 0) && !pulse;
    endproperty

    // Assertions
    assert property (pulse_at_max_count);
    assert property (pulse_only_at_max);
    assert property (counter_wrap);

```

```

    assert property (counter_increment);
    assert property (reset_behavior);

```

```
endmodule
```

7.10 Synthesis Considerations

7.10.1 Resource Utilization

WIDTH	Counter Bits	LUTs	FFs	Max Freq
8	3	8	4	500MHz
16	4	12	5	450MHz
32	5	18	6	400MHz
1024	10	28	11	350MHz

7.10.2 Timing Optimization

```

// For high-frequency applications, pipeline the comparison
module clock_pulse_pipelined #(
    parameter int WIDTH = 1000
) (
    input logic clk,
    input logic rst_n,
    output logic pulse
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;
    logic compare_result;

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

    // Pipeline the comparison
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            compare_result <= 1'b0;
        end else begin
            compare_result <= (r_counter == w_width_minus_one);
        end
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else begin

```

```

        if (compare_result)
            r_counter <= 'b0;
        else
            r_counter <= r_counter + 1;

        pulse <= compare_result;
    end
end
endmodule

```

7.11 Common Applications Summary

1. **Timing References:** System heartbeats, watchdog timers
2. **Sampling Systems:** ADC triggers, data acquisition timing
3. **Communication:** Baud rate generation, protocol timing
4. **Memory Systems:** Refresh timing, access scheduling
5. **Test Equipment:** Pattern generation, stimulus timing
6. **Power Management:** Activity monitoring, timeout generation
7. **Display Systems:** Refresh rates, sync generation

7.12 Design Guidelines

1. **Choose Appropriate WIDTH:** Balance between resolution and resource usage
2. **Consider Reset Timing:** Ensure clean startup behavior
3. **Validate Timing:** Verify pulse frequency matches requirements
4. **Plan for Variation:** Consider process, voltage, temperature effects
5. **Monitor Resource Usage:** Large counters can impact timing and area
6. **Use Enables:** Add enable signals for conditional operation
7. **Document Timing:** Clearly specify pulse rates and relationships

The clock pulse generator provides a simple, reliable solution for periodic timing generation, essential for many digital system functions requiring precise, regular timing events.

7.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

8 APB Slave CDC

Module: apb_slave_cdc.sv **Location:** rtl/amba/apb/ **Status:** ✓ Production Ready

8.1 Overview

The APB Slave CDC (Clock Domain Crossing) module provides a complete APB slave interface with integrated clock domain crossing between the APB (pclk) domain and an AXI/GAXI (aclk) domain. This enables safe integration of APB peripherals running at different clock frequencies.

8.1.1 Key Features

- ✓ **Safe CDC:** Gray-pointer asynchronous FIFOs, one per direction
- ✓ **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- ✓ **Independent Resets:** Either domain may be reset alone without corrupting the link
- ✓ **Full APB4 Support:** Complete AMBA 4 APB protocol compliance
- ✓ **Command/Response Interface:** Clean GAXI-style backend interface
- ✓ **Buffered Operation:** Integrated skid buffers for elastic storage

Protocol scope: APB4 only. For APB5 signalling use apb5_slave_cdc from rtl/amba/apb5/ – see the [APB5 book](#).

8.2 Module Interface

```
module apb_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH        = 2,
    // DEPRECATED / NO EFFECT -- retained only so existing
instantiations
    // still elaborate. See "CDC Implementation" below.
    parameter bit USE_2_PHASE_CDC = 1'b1
) (
    // Clock and Reset
    input  logic          aclk,
    input  logic          aresetn,
    input  logic          pclk,
    input  logic          presetn,
```

```

// APB Slave Interface (pclk domain)
input  logic          s_apb_PSEL,
input  logic          s_apb_PENABLE,
output logic          s_apb_PREADY,
input  logic [AW-1:0] s_apb_PADDR,
input  logic          s_apb_PWRITE,
input  logic [DW-1:0] s_apb_PWDATA,
input  logic [SW-1:0] s_apb_PSTRB,
input  logic [PW-1:0] s_apb_PPROT,
output logic [DW-1:0] s_apb_PRDATA,
output logic          s_apb_PSLVERR,

// Command Interface (aclk domain)
output logic          cmd_valid,
input  logic          cmd_ready,
output logic          cmd_pwrite,
output logic [AW-1:0] cmd_paddr,
output logic [DW-1:0] cmd_pwdata,
output logic [SW-1:0] cmd_pstrb,
output logic [PW-1:0] cmd_pprot,

// Response Interface (aclk domain)
input  logic          rsp_valid,
output logic          rsp_ready,
input  logic [DW-1:0] rsp_prdata,
input  logic          rsp_pslverr
);

```

8.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid-buffer depth in entries inside the wrapped apb_slave; also

Parameter	Type	Default	Description
USE_2_PHASE_CDC	bit	1	the floor for the CDC FIFO depth Deprecated and ignored. Has no effect on the generated logic

8.4 Clock Domains

8.4.1 APB Domain (pclk)

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

8.4.2 AXI Domain (aclk)

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

8.5 CDC Implementation

8.5.1 Structure

The module is an apb_slave in the pclk domain plus two independent asynchronous FIFOs:

FIFO	Direction	Write domain	Read domain	Payload width
u_cmd_cdc_fifo	Command	pclk / presetn	aclk / aresetn	$CPW = AW + DW + SW + PW + 1$
u_rsp_cdc_fifo	Response	aclk / aresetn	pclk / presetn	$RPW = DW + 1$

Both are gaxi_fifo_async instances with:

- **Pointer encoding:** gray-coded absolute read/write pointers
- **Synchronizer depth:** $N_FLOP_CROSS = 2$ (two-flop synchronizer per crossed pointer)

- **FIFO depth:** `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` – a floor of 4 entries regardless of the DEPTH used for the internal skid buffers. Powers of two are preferred for the gray-pointer encoding.

There is no separate metastability-hardening option; two-flop synchronization is fixed at instantiation.

8.5.2 Maximum Clock Ratio

There is no maximum ratio between `pclk` and `acclk`. Gray-pointer FIFOs impose no relationship between the two clocks – either may be arbitrarily faster, slower, or phase-unrelated, and either may be stopped indefinitely. Stopping `acclk` simply stalls the command FIFO’s read side; the APB side backpressures via `PREADY` held low and no data is lost.

Throughput, not correctness, is what the ratio affects: each transfer costs the usual two-flop synchronizer latency in each direction (roughly 2-3 destination clock edges per crossing), so a very slow `acclk` directly lengthens APB wait states.

8.5.3 Reset Behavior

`gaxi_fifo_async` resets each domain’s own pointer **and** that domain’s crossed copy of the remote pointer from that domain’s local reset. Resetting one side in isolation therefore leaves that side’s view self-consistent (both pointers zero, i.e. empty), and because the pointers are absolute positions rather than parity state, an independent reset cannot fabricate or swallow a transfer.

This is load-bearing: `presetn` and `aresetn` are genuinely separate reset domains in real integrations. For example, the `ddr2-char` harness pulses only the core-side reset on `CTRL.soft_reset` while the APB side stays up.

8.5.4 Why Not the Previous 2-Phase Handshake

`USE_2_PHASE_CDC` selected a toggle-based handshake in an earlier revision. It is now deprecated and ignored, and the parameter is retained only for source-compatibility with existing instantiations.

A 2-phase handshake encodes each transfer as a **toggle**. If the two domains are reset independently, the toggle parity desynchronizes and the link fabricates or drops exactly one transfer – permanently, because nothing ever re-syncs it. Paired with the `apb_slave` FSM, which pairs commands and responses by position rather than by tag, a single phantom transfer offsets the response stream by one forever: every read returns the previous read’s data.

This was observed on the Nexys A7 `ddr2-char` board on 2026-07-19. Reading a single CSR eight times returned the previous register’s value about three times before settling, while the non-CDC harness window was stable. The two mitigations now in place are the gray-pointer FIFOs described above and the orphan-response guard in `apb_slave` – see [apb_slave.md](#).

8.5.5 Timing Constraints

The crossed signals are the gray-coded pointers and the FIFO memory read path. Standard practice applies:

- Declare `pclk` and `aclk` asynchronous to each other (`set_clock_groups - asynchronous`).
- Do not over-constrain the pointer synchronizer paths; the gray encoding tolerates a one-bit-at-a-time skew by construction.

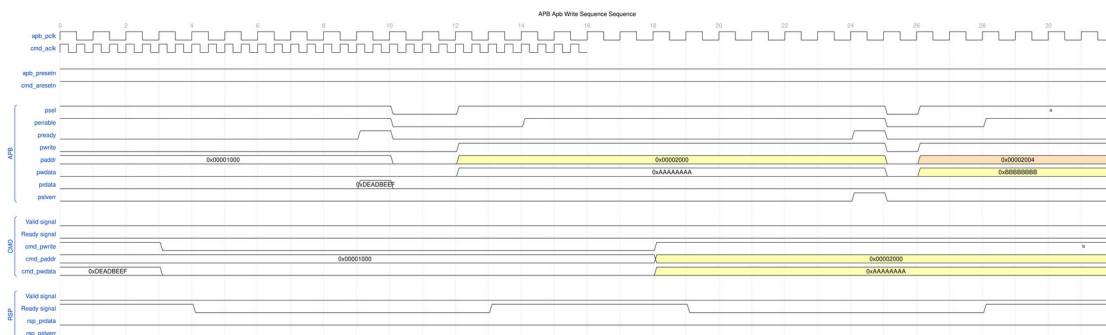
8.6 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

8.6.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from `pclk` domain to `aclk` domain:



Write CDC

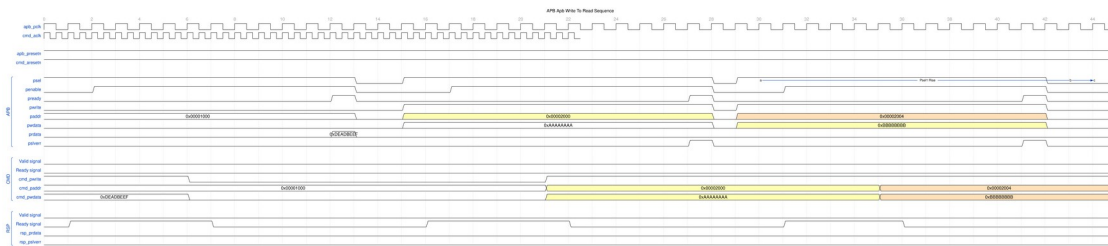
WaveJSON: [apb_write_sequence_001.json](#)

Key Observations: - APB transaction in `pclk` domain - CMD transaction crosses to `aclk` domain - Note the CDC latency between domains

8.6.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from `aclk` to `pclk` domain:

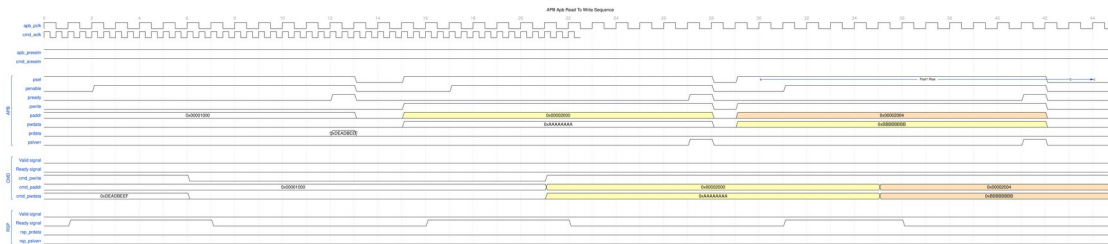
8.6.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: [apb_write_to_read_001.json](#)

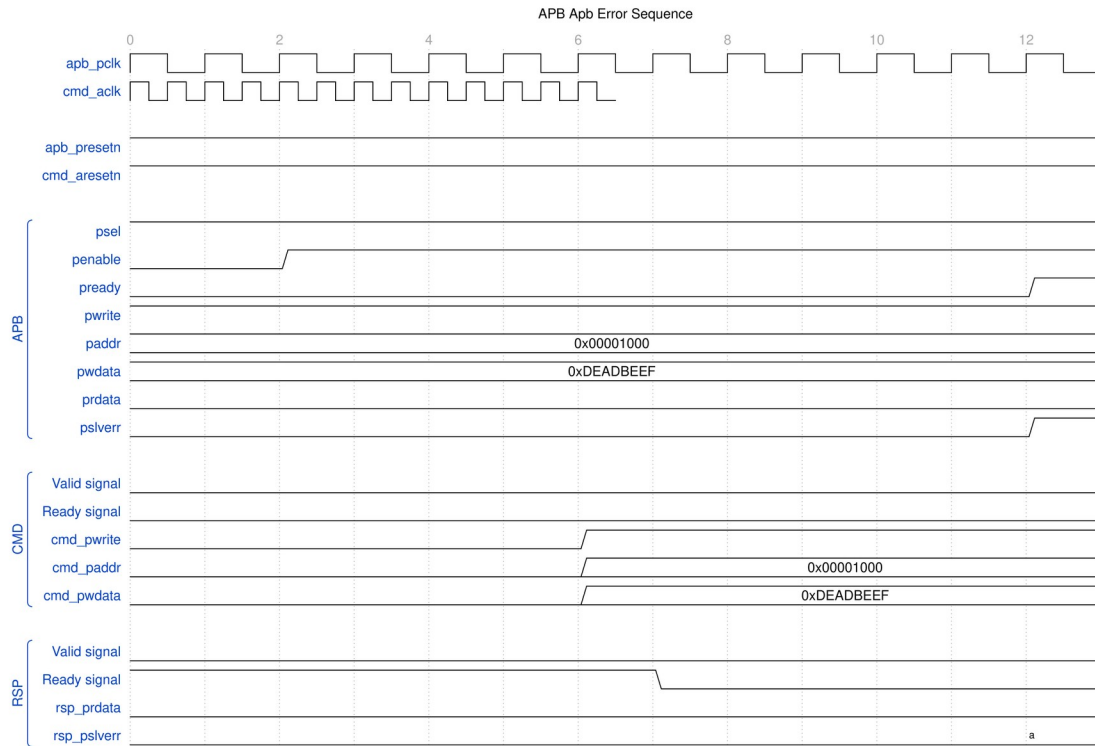
8.6.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: [apb_read_to_write_001.json](#)

8.6.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: [apb_error_001.json](#)

8.7 Usage Example

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk      (apb_clk),
    .presetn   (apb_resetn),

    // AXI clock domain
    .aclk      (axi_clk),
    .aresetn   (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL (apb_psel),
```

```

.s_apb_PENABLE (apb_penable),
.s_apb_PREADY  (apb_pready),
.s_apb_PADDR   (apb_paddr),
.s_apb_PWRITE  (apb_pwrite),
.s_apb_PWDATA  (apb_pwdata),
.s_apb_PSTRB   (apb_pstrb),
.s_apb_PPROT   (apb_pprot),
.s_apb_PRDATA  (apb_prdata),
.s_apb_PSLVERR (apb_pslverr),

// Command interface (aclk domain)
.cmd_valid      (cmd_valid),
.cmd_ready      (cmd_ready),
.cmd_pwrite     (cmd_pwrite),
.cmd_paddr      (cmd_paddr),
.cmd_pwdata     (cmd_pwdata),
.cmd_pstrb      (cmd_pstrb),
.cmd_pprot      (cmd_pprot),

// Response interface (aclk domain)
.rsp_valid      (rsp_valid),
.rsp_ready      (rsp_ready),
.rsp_prdata     (rsp_prdata),
.rsp_pslverr    (rsp_pslverr)
);

```

8.8 References

- **APB Slave:** [apb_slave.md](#)
 - **Clock-Gated Variant:** [apb_slave_cdc_cg.md](#)
 - **APB5 Equivalent:** [apb5_slave_cdc.md](#)
 - **CDC FIFO:** rtl/amba/gaxi/gaxi_fifo_async.sv
 - **Source:** rtl/amba/apb/apb_slave_cdc.sv
 - **Tests:** val/amba/test_apb_slave_cdc.py
 - **WaveDrom Test:**
val/amba/test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom
-

Last Updated: 2026-07-19

8.9 Navigation

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

9 APB Slave with CDC (Clock-Gated)

Module: `apb_slave_cdc_cg.sv` **Base Module:** [apb_slave_cdc](#) **Location:** `rtl/amba/apb/`
Status: ✓ Production Ready

9.1 Quick Reference

This is the **clock-gated variant** of [apb_slave_cdc](#).

For the clock-gating architecture and the underlying gate cell, see:

→ [Clock-Gated Variants Guide](#) and → [amba_clock_gate_ctrl](#)

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

9.2 Summary

The `apb_slave_cdc_cg` module adds power optimization to `apb_slave_cdc` through activity-based clock gating. **Both clock domains are gated independently** – the module instantiates two `amba_clock_gate_ctrl` blocks, one on `pclk` and one on `aclk`, sharing the same configuration inputs.

Structurally it is a sibling of `apb_slave_cdc` rather than a wrapper around it: it re-instantiates the same `apb_slave` plus the same pair of `gaxi_fifo_async` CDC FIFOs, but drives them from the gated clocks. The CDC behaviour described in [apb_slave_cdc.md](#) – gray pointers, `N_FLOP_CROSS=2`, independent-reset safety, no maximum clock ratio – applies unchanged.

While a domain is gated, that domain's ready outputs are forced low so no handshake can complete against a stopped clock.

- ✓ **Same Functionality:** 100% equivalent to base module
- ✓ **Dual-Domain Gating:** Separate gate cell per clock domain
- ✓ **Runtime Control:** Gating is enabled and tuned by input signals, not parameters

- ✓ **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

9.3 Additional Parameters

In addition to all [apb_slave_cdc](#) parameters:

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown counter, shared by both domains

USE_2_PHASE_CDC is inherited from the base module and is likewise **deprecated and ignored**. There is no ENABLE_CLOCK_GATING parameter and no per-domain CG_GATE_* parameters.

9.4 Additional Ports

In addition to all [apb_slave_cdc](#) ports:

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Global clock-gate enable for both domains. 0 = never gate
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating
pclk_cg_gating	1	Output	Asserted while the pclk domain is gated
pclk_cg_idle	1	Output	Asserted while the pclk domain buffers are empty
aclk_cg_gating	1	Output	Asserted while the aclk domain is gated
aclk_cg_idle	1	Output	Asserted while the aclk domain buffers are empty

9.5 Quick Usage

```
apb_slave_cdc_cg #(
    // Base module parameters (see apb_slave_cdc.md)
    .ADDR_WIDTH      (32),
    .DATA_WIDTH      (32),
    .DEPTH            (2),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk             (apb_clk),
    .presetn          (apb_resetn),
    .aclk             (core_clk),
    .aresetn          (core_resetn),

    // Clock gating control (applies to both domains)
    .cfg_cg_enable    (1'b1),
    .cfg_cg_idle_count (4'd8),

    // Per-domain gating status
    .pclk_cg_gating   (pclk_gated),
    .pclk_cg_idle     (pclk_idle),
    .aclk_cg_gating   (aclk_gated),
    .aclk_cg_idle     (aclk_idle),

    // ... all other ports same as apb_slave_cdc
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging CDC waveforms. Two independently gated clocks make cross-domain timing very hard to read.

9.6 Documentation

- **Base Module Functionality:** [apb_slave_cdc.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

9.7 Navigation

- [← Back to APB Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

10 APB5 Slave (Clock Domain Crossing)

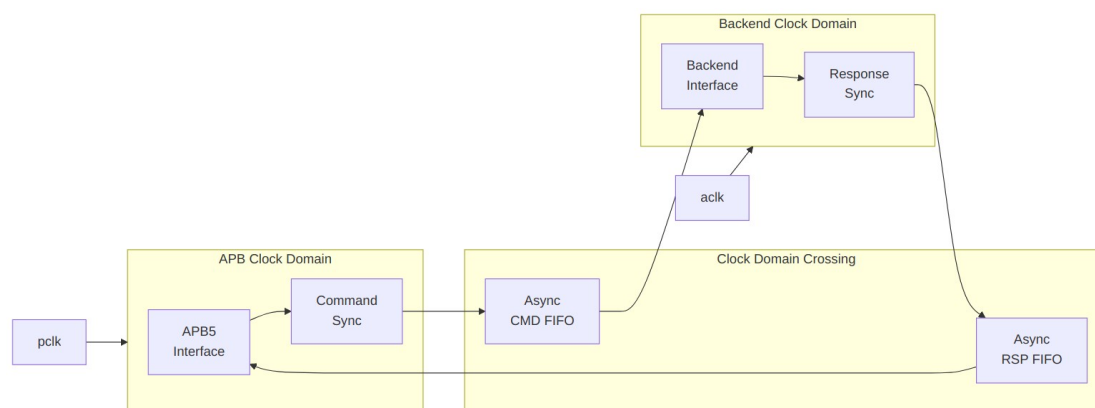
Module: apb5_slave_cdc.sv **Location:** rtl/amba/apb5/ **Status:** Production Ready

10.1 Overview

The APB5 Slave CDC module provides clock domain crossing capability between an APB5 bus clock domain and a backend clock domain. It safely transfers APB5 transactions across asynchronous clock boundaries.

10.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing (Gray-coded pointers)
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- Wake-up request crossing via a dedicated level synchronizer
- Single DEPTH parameter sizes both directions
- Metastability protection with 2-flop pointer synchronizers



Module Architecture

10.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Skid-buffer depth of the wrapped apb5_slave; one of {2, 4, 6, 8}
ENABLE_PARITY	bit	0	Enable parity generation and checking
USE_2_PHASE_CDC	bit	1	Deprecated and ignored – retained for source compatibility

There is no CMD_DEPTH, RSP_DEPTH or SYNC_STAGES parameter on this module. The two asynchronous FIFOs are sized internally from DEPTH:

```
localparam int CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH;
```

so the CDC FIFOs are never shallower than 4 entries regardless of DEPTH. The synchronizer depth is fixed at 2 flops (N_FLOP_CROSS(2) on both gaxi_fifo_async instances) and is not exposed as a parameter. A design needing 3-flop synchronization for an extreme clock ratio must change the instantiation in the RTL.

10.3 Ports

10.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	Backend/user clock
aresetn	1	Input	Backend/user reset (active low)

The backend clock and reset are named `aclk` and `aresetn`, matching the rest of the AMBA library. There are no `bclk/bresetn` ports.

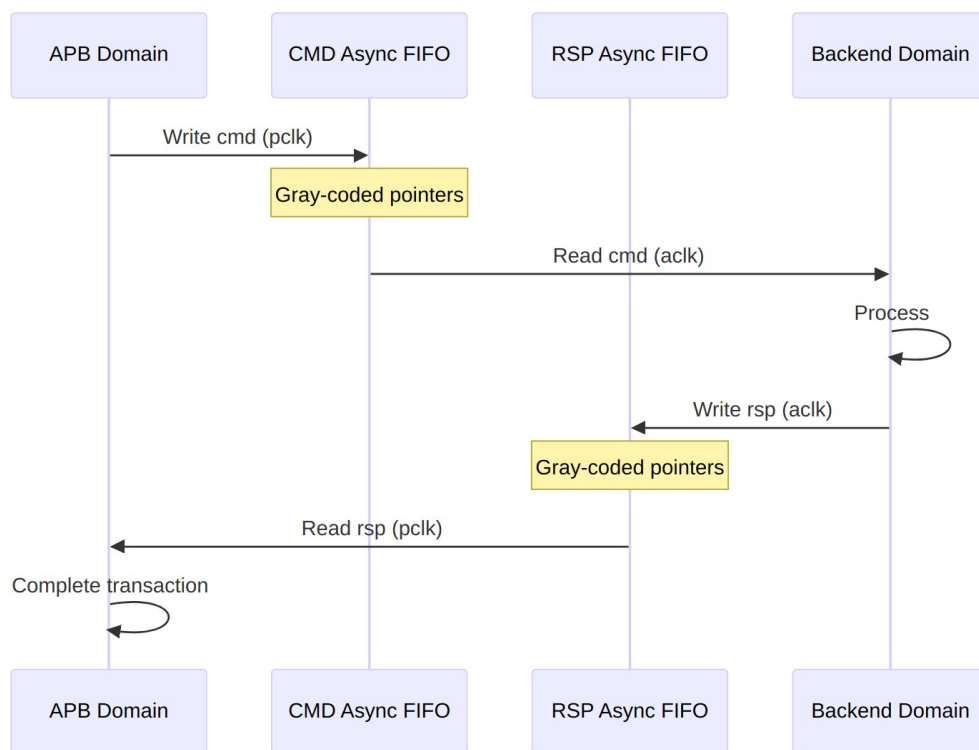
10.3.2 APB5 Slave Interface

Same as [apb5_slave](#) - operates in `pclk` domain, including the optional parity signals.

10.3.3 Backend Interface

Same command/response interface as [apb5_slave](#) - operates in the `aclk` domain. `wakeup_request`, `parity_error_wdata` and `parity_error_ctrl` are also present.

10.4 Clock Domain Crossing



CDC Mechanism

10.4.1 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- aclk (different frequency)
- APB transaction signals
- FIFO write/read pointers
- Backend transaction signals
- CDC latency

10.5 Usage Example

```
apb5_slave_cdc #(  
    .ADDR_WIDTH    (32),  
    .DATA_WIDTH    (32),  
    .AUSER_WIDTH   (4),
```

```

        .WUSER_WIDTH      (4),
        .RUSER_WIDTH      (4),
        .BUSER_WIDTH      (4),
        .DEPTH            (2),
        .ENABLE_PARITY    (0)
    ) u_apb5_slave_cdc (
        // APB clock domain
        .pclk              (apb_clk),
        .presetn            (apb_rst_n),

        // Backend clock domain
        .aclk              (backend_clk),
        .aresetn            (backend_rst_n),

        // APB5 slave interface (pclk domain)
        .s_apb_PSEL        (s_apb_psel),
        .s_apb_PENABLE     (s_apb_penable),
        // ... other APB signals

        // Backend interface (aclk domain)
        .cmd_valid          (backend_cmd_valid),
        .cmd_ready          (backend_cmd_ready),
        // ... other backend signals

        // Wake-up request (aclk domain, synchronized internally to pclk)
        .wakeup_request     (backend_wakeup)
    );

```

10.6 Design Notes

10.6.1 CDC Mechanism

Both directions cross through `gaxi_fifo_async` instances with Gray-coded pointers and a fixed 2-flop synchronizer (`N_FLOP_CROSS(2)`) on each pointer crossing. The command FIFO is written in `pclk` and read in `aclk`; the response FIFO is written in `aclk` and read in `pclk`.

The `wakeup_request` input is the one signal that does not go through a FIFO: it crosses from `aclk` into `pclk` through a `cdc_synchronizer` before reaching the wrapped `apb5_slave`. Because it is a level, not a pulse, this is safe – but it means `wakeup_request` must be held asserted long enough to be sampled in the `pclk` domain (at least two `pclk` periods).

10.6.2 FIFO Depth Sizing

- DEPTH sizes the apb5_slave skid buffers; the CDC FIFOs are sized separately as $\max(\text{DEPTH}, 4)$, so the crossing itself is never shallower than 4 entries even at the default DEPTH=2
- Deeper FIFOs buy tolerance for a slow backend or a bursty APB master; they do not reduce the per-transfer crossing latency
- Because APB is single-outstanding, depth beyond a handful of entries has little effect on throughput for this module

10.6.3 Reset Synchronization

presetn and aresetn are independent reset domains and either may be asserted alone. gaxi_fifo_async resets each domain's own pointer *and* that domain's crossed copy of the remote pointer from the local reset, so a one-sided reset leaves that side self-consistent (both pointers zero, meaning empty) instead of fabricating or swallowing a transfer. Pointers are absolute positions rather than toggle parity, which is what makes the one-sided case safe.

The practical consequence: resetting only the backend (aresetn) discards in-flight commands from that side's point of view while the APB side stays up. A transfer already accepted on the APB side but not yet read out will not be delivered, so the APB master may see a transaction time out. Integrators who pulse only one reset should quiesce the bus first.

Neither reset is internally synchronized to the other domain's clock; each is expected to be already synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator.

10.6.4 Latency

- Crossing latency is the async-FIFO write-to-read pointer synchronization: on the order of 2-3 destination-clock cycles per direction, plus the source-clock cycle that performs the write
 - A full APB transfer therefore costs the command crossing plus the response crossing before PREADY can assert
 - Additional latency if the FIFOs are full (backpressure) or the backend is slow
-

10.7 Related Documentation

- [APB5 Slave](#) - Base slave without CDC
- [APB5 Slave CDC CG](#) - CDC with clock gating
- [GAXI Async FIFO](#) - The async FIFO used for both crossings

- [Clock Domain Crossing](#) - CDC design patterns and reset behavior
-

10.8 Navigation

- [← Back to APB5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

11 APB5 Slave (CDC + Clock-Gated)

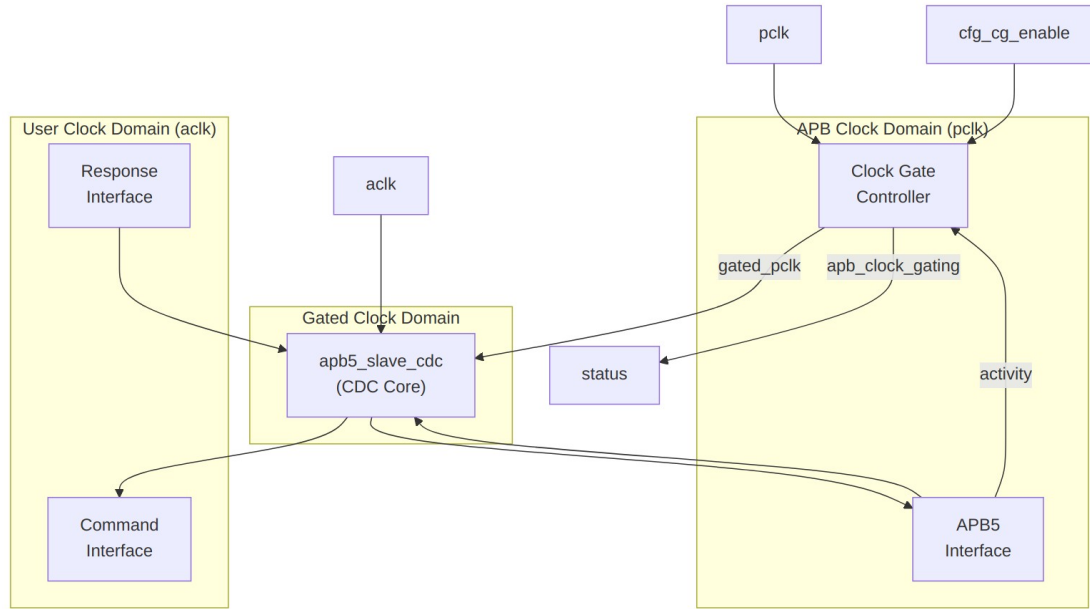
Module: apb5_slave_cdc_cg.sv **Location:** rtl/amba/apb5/ **Status:** Production Ready

11.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps apb5_slave_cdc with clock gating control to reduce power consumption during idle periods while maintaining safe operation across asynchronous clock boundaries.

11.1.1 Key Features

- Full APB5 protocol support with all extensions
 - Asynchronous clock domain crossing (APB to backend)
 - Clock gating for power reduction during idle
 - All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
 - PWAKEUP signal handling across domains
 - Optional parity support for data integrity
 - Automatic wake-up on transaction activity
-



Module Architecture

11.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
DEPTH	int	2	Skid-buffer depth of the wrapped slave; CDC FIFOs use max(DEPTH, 4)
ENABLE_PARITY	bit	0	Enable parity generation

Parameter	Type	Default	Description
			and checking
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = $2^N - 1$ cycles)
USE_2_PHASE_CDC	bit	1	Deprecated and ignored – retained for source compatibility

As with [apb5_slave_cdc](#), there is no SYNC_STAGES parameter: pointer synchronization is fixed at 2 flops inside the async FIFOs.

11.3 Ports

11.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

11.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

11.3.3 APB5 Slave Interface

Same as [apb5_slave](#) - operates in pclk domain.

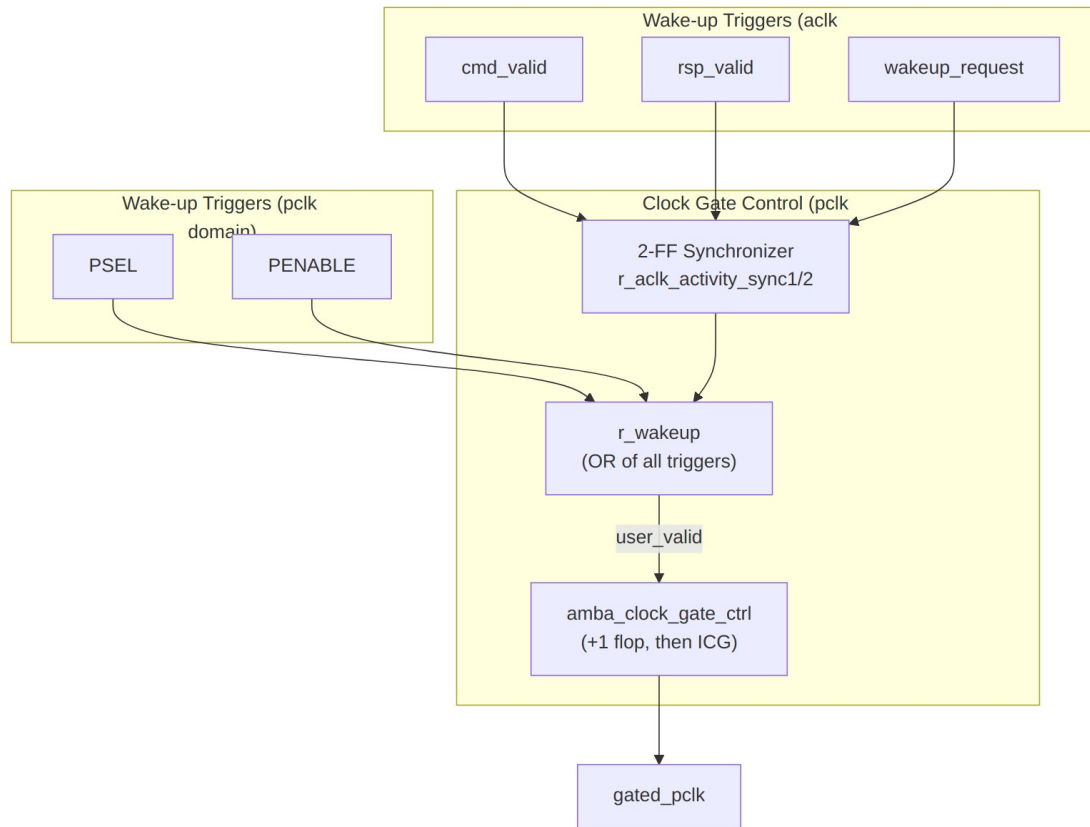
11.3.4 Backend Interface

Same command/response interface as [apb5_slave_cdc](#) - operates in `ac1k` domain.

11.3.5 Status Outputs

Port	Width	Direction	Description
apb_clock_gating	1	Output	Indicates clock is currently gated
parity_error_write_data	1	Output	Write data parity error detected
parity_error_control	1	Output	Control signal parity error

11.4 Clock Gating and CDC Interaction



Wake-up Logic

The `aclk`-domain activity terms are ORed together and passed through a two-flop synchronizer before being combined with the `pclk`-domain terms. Without that synchronizer these signals would cross domains unsynchronized.

11.4.1 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- `pclk` (ungated)
- `gated_pclk`
- `aclk` (different frequency)
- `s_apb_PSEL` (wake trigger)
- `apb_clock_gating` indicator
- Transaction flow across CDC with gating
- Wake-up latency from `PSEL` to clock active

11.5 Usage Example

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH      (32),
    .DATA_WIDTH      (32),
    .AUSER_WIDTH     (4),
    .WUSER_WIDTH     (4),
    .RUSER_WIDTH     (4),
    .BUSER_WIDTH     (4),
    .DEPTH           (2),
    .ENABLE_PARITY    (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk          (apb_clk),
    .presetn        (apb_rst_n),

    // User clock domain
    .aclk           (user_clk),
    .aresetn        (user_rst_n),

    // Clock gating
    .cfg_cg_enable  (1'b1),
    .cfg_cg_idle_count (4'd8),
    .apb_clock_gating (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL      (s_apb_psel),
    .s_apb_PENABLE    (s_apb_penable),
    .s_apb_PREADY     (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid        (backend_cmd_valid),
    .cmd_ready        (backend_cmd_ready),
    // ... other command signals

    .rsp_valid        (backend_rsp_valid),
    .rsp_ready        (backend_rsp_ready),
    // ... other response signals

    // Wake-up control (aclk domain)
    .wakeup_request    (backend_wakeup)
);
```

11.6 Design Notes

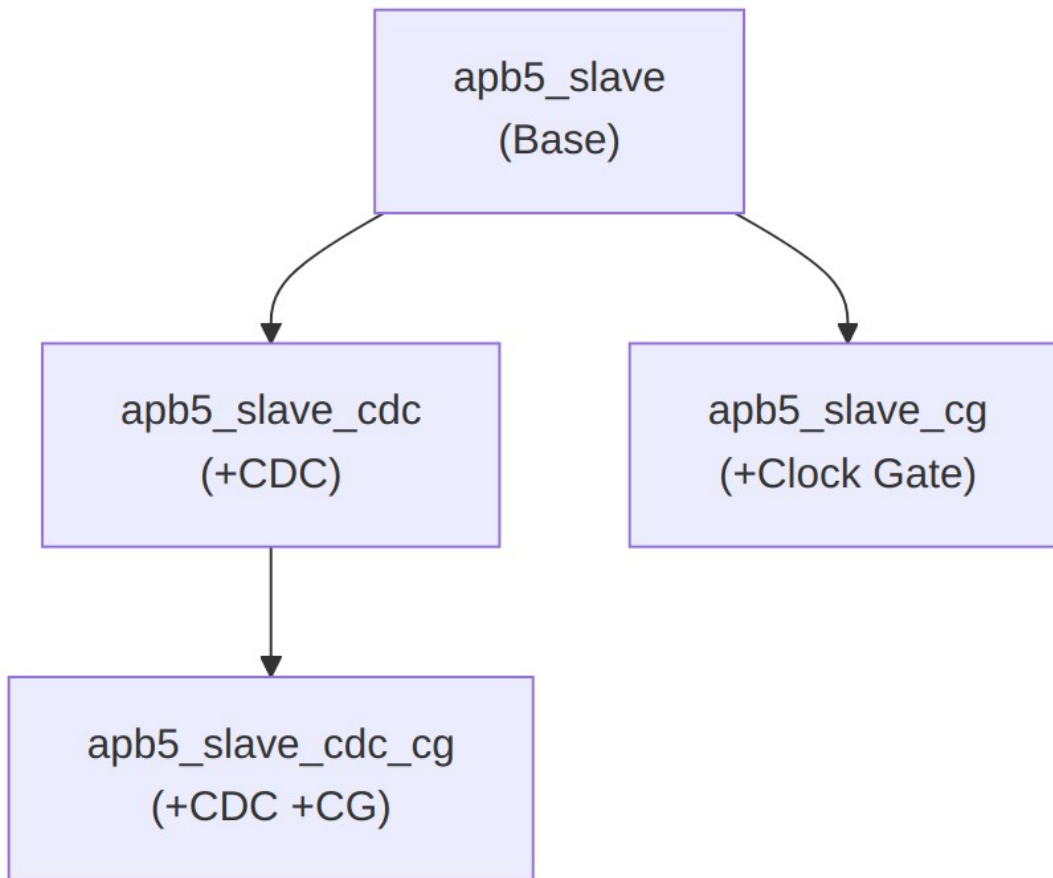
11.6.1 Power and Latency Trade-offs

Wake-up is **registered, not combinational**. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. A pclk-domain trigger (PSEL or PENABLE) passes through this wrapper's r_wakeup and then through the flop inside amba_clock_gate_ctrl, so APB5 is a two-stage family and the first usable gated pclk rising edge arrives **3 ungated pclk cycles** after the trigger. An aclk-domain trigger (cmd_valid, rsp_valid, wakeup_request) first crosses a 2-flop synchronizer into pclk, adding about two more pclk cycles.

Configuration	Power Savings	Wake-up Latency from PSEL
CG disabled	None	0 cycles (clock always running)
CG idle=4	Moderate	2 register stages; first usable edge 3 ungated pclk cycles
CG idle=16	Good	2 register stages; first usable edge 3 ungated pclk cycles

The wake-up latency does not depend on cfg_cg_idle_count; the idle count only controls how long the block waits before gating. Latency is absorbed as APB wait states, since the slave holds PREADY low until it is running again.

Deliberately keeping the wake-up path registered avoids driving a clock-gate enable from a combinational function of a bus input. The gating element itself is an ICG cell instantiated by clock_gate_ctrl, which is glitch-free by construction. ICG cells are an ASIC library primitive; FPGA targets should use a clock-enable approach instead.



Combined Feature Hierarchy

11.6.2 Reset Considerations

- APB domain reset (presetn) controls the APB interface, the wake-up synchronizer and the clock gate
 - User domain reset (aresetn) controls the backend side of the CDC FIFOs
 - Each reset must already be synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator; this module does not synchronize either reset into the other domain
 - The async FIFOs tolerate a one-sided reset without corrupting the pointer state – see [apb5_slave_cdc](#) for the mechanism and for the in-flight transaction caveat
-

11.7 Related Documentation

- [APB5 Slave](#) - Base slave module
 - [APB5 Slave CDC](#) - CDC variant without clock gating
 - [APB5 Slave CG](#) - Clock gating without CDC
-

11.8 Navigation

- [← Back to APB5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)